Jak czytać wyniki modeli ML

Dane i trening · wyniki na danych testowych · gdzie patrzeć w notebooku

ML-Ergebnisse richtig lesen

Daten & Training · Ergebnisse auf Testdaten · wo man im Notebook hinschaut

#	NOTEBOOK	MODEL / MODELL	DANE / DATEN (WIERSZE / ZEILEN)
1	`3-Lineare_Regression-Projekt_Uebung-Loesung`	LinearRegression	Ecommerce Customers (500)
2	`3-Logistische_Regression_Projekt-Loesung`	LogisticRegression	advertising.csv (1 000)
3	`3-K_Nearest_Neighbors_Projekt-Loesung`	KNeighborsClassifier	KNN_Project_Data (~1 000)
4	`3-Decision_Trees_und_Random_Forests_Projekt-Loesung`	DecisionTree, RandomForest	loan_data.csv (9 578)
5	`3-Support_Vector_Machines_Projekt-Loesung`	SVC + GridSearchCV	Iris (150)
6	`3-K_Means_Clustering_Projekt-Loesung`	KMeans	College_Data (777)
7	`3-Nlp_Projekt-Loesung`	CountVectorizer/TF-IDF + MultinomialNB	Yelp (10 000)
8	`04-Keras-Projekt-Uebung-Loesung`	Keras Sequential (Dense)	LendingClub (396 030)

CZĘŚĆ A
po polsku
od strony 3

TEIL B
auf Deutsch
ab Seite 28

✓ w porządku ! z zastrzeżeniami ✗ problem
obliczone liczba policzona z wyników notebooka · dodatkowy test własne uruchomienie na tych samych danych

✓ in Ordnung ! mit Einschränkungen ✗ Problem
berechnet aus den Notebook-Ausgaben berechnet · Zusatztest eigener Lauf mit denselben Daten

Stan / Stand: 16.10.2026 · Podstawa: wyniki zapisane w 8 notebookach (wersje przesłane 16.10.2026) / Grundlage: die in den 8 Notebooks gespeicherten Ausgaben

Spis treści · Inhalt

SPIS TREŚCI — CZĘŚĆ A (POLSKI)

A0	Jak czytać ten dokument + ściąga	3
A1	Regresja liniowa — klienci e-commerce	5
A2	Regresja logistyczna — kliknięcia w reklamę	7
A3	KNN — dane zanonimizowane	10
A4	Drzewo decyzyjne i las losowy — LendingClub	12
A5	SVM — irysy	14
A6	K-Means — uczelnie w USA	16
A7	NLP — recenzje Yelp	19
A8	Keras — LendingClub (sieć neuronowa)	21
A9	Porównanie wszystkich modeli i najważniejsze lekcje	26
A10	Uwagi techniczne przy ponownym uruchamianiu	27

INHALT — TEIL B (DEUTSCH)

B0	So liest du dieses Dokument + Spickzettel	28
B1	Lineare Regression — E-Commerce-Kunden	30
B2	Logistische Regression — Klicks auf Werbung	32
B3	KNN — anonymisierte Daten	35
B4	Entscheidungsbaum und Random Forest — LendingClub	37
B5	SVM — Iris	39
B6	K-Means — US-Hochschulen	41
B7	NLP — Yelp-Bewertungen	44
B8	Keras — LendingClub (neuronales Netz)	46
B9	Alle Modelle im Vergleich + wichtigste Lektionen	51
B10	Technische Hinweise für neue Bibliotheksversionen	52

Jak korzystać

Każdy rozdział A1–A8 ma tę samą budowę: metryczka → „W skrócie" → 1) dane i trening → 2) wyniki na danych testowych → na co uważać → wniosek. Odnośniki `In [n]` wskazują komórkę w notebooku. Ściąga z metrykami: A0.

So benutzt du es

Jedes Kapitel B1–B8 ist gleich aufgebaut: Steckbrief → „Kurz gesagt" → 1) Daten und Training → 2) Ergebnisse auf Testdaten → Vorsicht → Fazit. Verweise `In [n]` zeigen auf die Zelle im Notebook. Spickzettel mit Metriken: B0.

CZĘŚĆ A

po polsku

Interpretacja 8 notebooków: dane i trening · wyniki na danych testowych · gdzie patrzeć

A0 Jak czytać ten dokument + ściąga

Dla każdego z 8 notebooków odpowiadam na dwa pytania: **(1) czy dane nadawały się do treningu i jak przebiegł trening** oraz **(2) jak interpretować wyniki na danych testowych**, czyli na danych, których model nie widział podczas uczenia. Każdy wniosek ma odnośnik do miejsca w notebooku, w którym go widać.

Budowa każdego rozdziału

- **Metryczka** — plik, zadanie, model, typ problemu, dane.
- **W skrócie** — trzy oceny: Dane / Trening / Wynik testu (✓ w porządku, ! z zastrzeżeniami, ✗ problem).
- **1) Dane i trening — gdzie patrzeć** — wiersze „dowodów": po lewej komórka i kod, po prawej *co widać → co to znaczy*.
- **2) Wyniki na danych testowych — jak czytać** — macierz pomyłek, metryki, porównanie z baseline.
- **Na co uważać**, czasem **Dodatkowy test**, na końcu **Wniosek**.

Odnośniki i oznaczenia

`In [5]` to licznik wykonania widoczny po lewej stronie komórki w Jupyterze. Obok podaję fragment kodu, żebyś mógł znaleźć komórkę przez Ctrl+F. Po ponownym uruchomieniu notebooka numery się zmienią — wtedy szukaj po kodzie. W notebooku Keras numery się powtarzają (plik zawiera wyniki z dwóch przebiegów), więc tam ważniejszy jest fragment kodu.

obliczone — liczby nie ma wprost w notebooku; policzyłem ją z widocznych wyników (np. baseline z kolumny `support`). dodatkowy test — własne uruchomienie kodu na tych samych, publicznie dostępnych danych (K-Means i NLP). Wyniki notebooków odtworzyły się w nim co do sztuki, więc to na pewno te same dane.

Ściąga 1 — czy dane nadają się do treningu?

PYTANIE	GDZIE PATRZEĆ	SYGNAŁ OSTRZEGAWCZY
Ile jest danych i czy są kompletne?	`df.info()` : liczba wierszy, *Non-Null Count*, *Dtype*	*Non-Null* mniejsze niż liczba wierszy = braki; `object` = tekst, który trzeba zakodować
Czy wartości są możliwe?	`df.describe()` : wiersze *min* i *max*	procent > 100, ujemne czasy lub kwoty, *max* wielokrotnie większe od 75%
Czy cechy mają podobną skalę?	`describe()` : *std*, *min*/*max*	różnice ×100–×1000 → skalowanie przy KNN, K-Means, SVM, sieciach
Czy klasy są zbalansowane?	`countplot` , `value_counts()` , średnia kolumny 0/1	jedna klasa ≥ 70–80% → accuracy myli, trzeba liczyć baseline
Czy cechy rozdzielają klasy?	`pairplot` , histogramy i boxploty z `hue=etykieta` , `corr()` z targetem	rozkłady klas prawie się pokrywają → słaby model
Czy nie ma przecieku danych?	lista cech; kolejność `fit` i `train_test_split`	cecha znana dopiero „po fakcie"; scaler / vectorizer / imputacja dopasowane przed podziałem

PYTANIE	GDZIE PATRZEĆ	SYGNAŁ OSTRZEGAWCZY
Czy ocena będzie uczciwa?	`train_test_split`: `test_size`, `random_state`, `stratify`	brak `random_state` (wyniki niepowtarzalne); test użyty do strojenia

Ściąga 2 — jak przebiegł trening?

- W scikit-learn `fit()` nic nie wypisuje. O jakości mówią dopiero wyniki. Szybki test: porównaj `model.score(X_train, y_train)` z `model.score(X_test, y_test)`. **Duża różnica** = przeuczenie (overfitting); **oba wyniki niskie** = niedouczenie (model za prosty albo w danych brak sygnału).
- Krzywe uczenia (Keras): *loss* i *val_loss* spadają razem → dobrze; *val_loss* stoi albo rośnie, a *loss* dalej spada → przeuczenie; oba wysokie → niedouczenie.
- Wykres błędu od hiperparametru (KNN): wybieraj wartość z płaskiego minimum, a nie z pojedynczego „dołka".

Ściąga 3 — jak czytać wyniki na danych testowych

	przewidziane 0	przewidziane 1
prawdziwe 0	**TN** trafione 0	**FP** fałszywy alarm
prawdziwe 1	**FN** przeoczenie	**TP** trafione 1

W scikit-learn **wiersze = prawda, kolumny = przewidywanie**. Klasy są ułożone według posortowanych etykiet (0, 1 albo 1, 5 albo alfabetycznie). Przekątna = trafienia.

- **accuracy** = (TN + TP) / wszystkie
- **precision** (klasy 1) = TP / (TP + FP) — *gdy model mówi „1", jak często ma rację?*
- **recall** (klasy 1) = TP / (TP + FN) — *jaki odsetek prawdziwych „1" znalazł?*
- **F1** = 2·P·R / (P + R) — kompromis precision i recall
- **support** = liczba prawdziwych przykładów klasy w teście
- **macro avg** = zwykła średnia z klas (każda klasa tak samo ważna); **weighted avg** = średnia ważona supportem (zdominowana przez dużą klasę)
- **balanced accuracy** = średnia z recall wszystkich klas; przy 2 klasach 0,5 = zgadywanie

- **Baseline** = udział największej klasy w zbiorze testowym (największy *support* / suma). Tyle accuracy ma „model", który zawsze mówi to samo. Model jest coś wart dopiero wyraźnie powyżej tej liczby.
- **Niepewność** accuracy: ok. ±1,96·√(p·(1−p)/n). Dla n = 45 to ±6 p.p., n = 300: ±4, n = 1226: ±1,5, n = 7905: ±0,7. Mniejsze różnice to szum.
- **Regresja**: MAE (średni błąd w jednostkach targetu), MSE (kwadraty — mocniej karze duże błędy), RMSE (pierwiastek z MSE, znów w jednostkach targetu), R^2 (udział wyjaśnionej zmienności: 1 = idealnie, 0 = nie lepiej niż średnia). RMSE porównuj z odchyleniem std targetu. Reszty (prawda − przewidywanie) powinny być symetryczne wokół 0 i bez wzorca.
- **Klastrowanie**: numery klastrów są losowe. Z etykietami porównuje się przez najlepsze dopasowanie albo miarą niezależną od numeracji (Adjusted Rand Index).

KOLEJNOŚĆ CZYTANIA WYNIKU KLASYFIKACJI

1. Macierz pomyłek: czy każda klasa jest w ogóle przewidywana? (kolumna samych zer = nie)
2. Accuracy vs baseline.
3. Recall i precision najważniejszej klasy — zwykle tej mniejszej.
4. Macro avg / balanced accuracy.
5. Wielkość zbioru testowego (niepewność).
6. Dopiero potem wnioski biznesowe.

A1 Regresja liniowa — klienci e-commerce

Plik	3-Lineare_Regression-Projekt_Uebung-Loesung.ipynb
Zadanie	przewidzieć roczne wydatki klienta (`Yearly Amount Spent` , w $) na podstawie 4 cech zachowania
Model	`LinearRegression` (scikit-learn)
Typ	regresja (wynik to liczba, nie klasa)
Dane	500 klientów; 4 cechy liczbowe + 3 kolumny tekstowe (niewykorzystane)

W SKRÓCIE

✓ **Dane**	Bardzo dobre: kompletne, czyste, z wyraźnymi liniowymi zależnościami. Dane są wygenerowane sztucznie — stąd ich „idealność".
✓ **Trening**	Poprawny podział 70/30 z `random_state=101` . Prosty model (5 parametrów) na 350 przykładach — przeuczenie praktycznie wykluczone.
✓ **Wynik testu**	Bardzo dobry: MAE ≈ 7,23 $, RMSE ≈ 8,93 $ przy średnich wydatkach 499 $; R^2 ≈ 0,99 (obliczone). Reszty wyglądają na normalne.

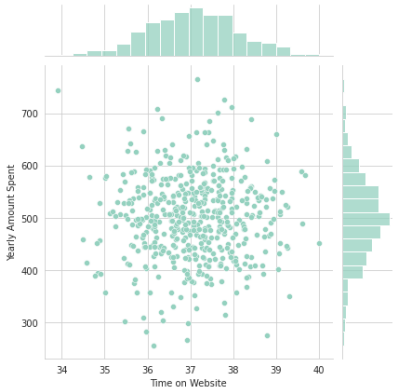
1) Dane i trening — gdzie patrzeć

`In [5]` `customers.info()`	**Widać:** 500 wierszy, w każdej kolumnie „500 non-null"; 5 kolumn `float64` i 3 `object` (Email, Address, Avatar). → **Wniosek:** brak braków danych. Kolumny tekstowe to identyfikatory i losowe atrybuty — słusznie nie trafiają do modelu (In [14] wybiera tylko 4 cechy liczbowe).
`In [4]` `customers.describe()`	**Widać:** cztery cechy mają *std* ≈ 1,0 (0,99 / 0,99 / 1,01 / 1,00), a średnia ≈ mediana (np. 33,05 vs 33,08). Zakresy są sensowne (Length of Membership 0,27–6,92 roku, brak wartości ujemnych). → **Wniosek:** rozkłady symetryczne, bez absurdalnych wartości. Tak równe *std* to znak danych wygenerowanych. Ważna konsekwencja: wszystkie cechy mają tę samą skalę, więc później wolno bezpośrednio porównywać współczynniki.
`In [7]` `jointplot(x='Time on Website', …)`	**Widać:** okrągła chmura punktów bez nachylenia. → **Wniosek:** czas na stronie www nie wiąże się z wydatkami. Komentarz w kodzie „*# Je mehr Zeit auf der Webseite, desto mehr Ausgaben*" przeczy wykresowi (i późniejszemu współczynnikowi 0,19). Zasada: wierz wykresom i liczbom, nie komentarzom.
`In [8]` `jointplot(x='Time on App', …)`	**Widać:** chmura wyraźnie nachylona w górę. → **Wniosek:** dłuższy czas w aplikacji idzie w parze z wyższymi wydatkami.
`In [10] / In [11]` `sns.pairplot(customers)`	**Widać:** w wierszu „Yearly Amount Spent" najwęższa, ukośna chmura jest przy Length of Membership; pary cech wejściowych tworzą okrągłe chmury. → **Wniosek:** najsilniejszy predyktor to długość członkostwa (tak samo odpowiada notebook w In [11]). Cechy nie są ze sobą skorelowane → brak współliniowości, współczynniki będą stabilne.
`In [12]` `lmplot(x='Length of Membership', …)`	**Widać:** punkty blisko linii, bardzo wąski cień przedziału ufności. → **Wniosek:** zależność jest liniowa — regresja liniowa to dobry wybór.
`In [16]` `train_test_split(X, y, test_size=0.3, random_state=101)`	**Widać:** 350 wierszy treningowych (potwierdza In [22] `print(X_train.head)` : „[350 rows x 4 columns]") i 150 testowych. → **Wniosek:** rozsądny podział; `random_state` sprawia, że wynik jest powtarzalny.

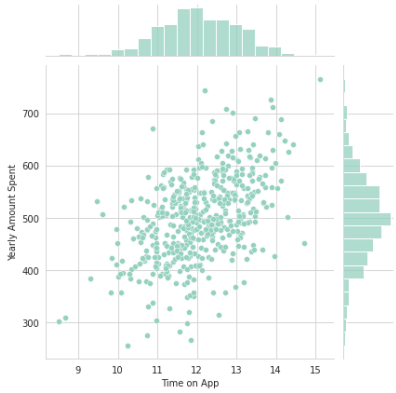
In [19]

```
lm.fit(X_train, y_train)
```

Widać: tylko `LinearRegression()` — sklearn nie wypisuje żadnych metryk treningu. → **Wniosek:** o przebiegu treningu wnioskujemy pośrednio: 5 parametrów na 350 przykładów = brak ryzyka przeuczenia. Sprawdzenie: `lm.score(X_train, y_train)` i `lm.score(X_test, y_test)` powinny dać prawie to samo R^2.



In [7] Time on Website vs. Yearly Amount Spent — chmura bez trendu: brak związku.



In [8] Time on App vs. Yearly Amount Spent — wyraźny trend rosnący.

2) Wyniki na danych testowych — jak czytać

In [22]

```
plt.scatter(y_test,
predictions)
```

Widać: punkty układają się niemal idealnie na przekątnej. → **Jak czytać:** każdy punkt to klient testowy: oś X — prawdziwe wydatki, oś Y — przewidziane. Idealny model = linia y = x. Wąski pas = małe błędy. Zakrzywienie oznaczałoby nieliniowość, „lejek" rozszerzający się w prawo — rosnący błąd dla dużych wartości. Tu nic z tych rzeczy.

In [23]

```
metrics.mean_absolute_err
or(…) /
mean_squared_error(…)
```

Widać: MAE 7,23; MSE 79,81; RMSE 8,93. → **Jak czytać:** MAE — przeciętna pomyłka 7,23 $; RMSE — „typowa" pomyłka 8,93 $ (mocniej karze duże błędy); MSE — to samo w $², trudne do interpretacji. Punkt odniesienia: średnie wydatki 499 $, std 79 $ (In [4]). Model, który zawsze podaje średnią, miałby RMSE ≈ 79 $ — nasz myli się ok. 9 razy mniej, a MAE to ok. 1,5% średnich wydatków. RMSE tylko trochę większe od MAE → brak pojedynczych ogromnych pomyłek.

MAE	RMSE	R^2 (szacunek)	Zbiór testowy
7,23 $	**8,93 $**	**≈ 0,99**	**150**
≈ 1,5% średniej	vs ≈ 79 $ dla „średniej"	obliczone	klientów

sekcja „Das Modell bewerten"

Widać: tekst zapowiada R^2 („erklärte Varianz"), ale kod go nie liczy. → **Wniosek:** szacunek obliczone: $R^2 \approx 1 - MSE/Var(y) = 1 - 79{,}8 / 79{,}3^2 \approx 0{,}987$, czyli model wyjaśnia ok. 99% zmienności wydatków. Dokładnie: `metrics.r2_score(y_test, predictions)`.

In [30]

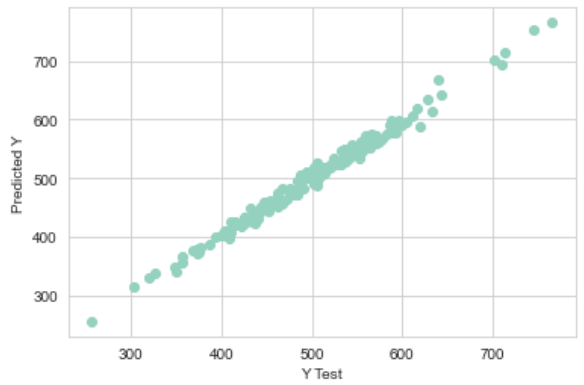
```
sns.histplot((y_test-
predictions), bins=50)
```

Widać: kształt dzwonu, środek przy 0, zakres ok. −27…+30 $. → **Jak czytać:** reszta = prawda − przewidywanie. Środek w 0 → model nie myli się systematycznie w żadną stronę; symetria i brak długich „ogonów" → założenia regresji liniowej spełnione. Warto dodać wykres reszta vs przewidywanie (powinien być bezkształtną chmurą).

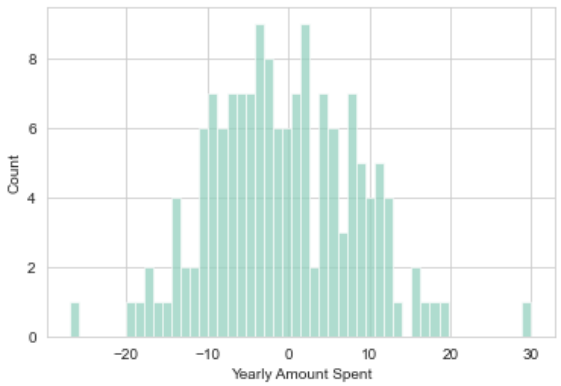
In [20] / In [25]

```
lm.coef_ /
pd.DataFrame(lm.coef_,
X.columns)
```

Widać: Length of Membership 61,28; Time on App 38,59; Avg. Session Length 25,98; Time on Website 0,19. → **Jak czytać:** przy pozostałych cechach bez zmian: +1 rok członkostwa ≈ +61 $ rocznie, +1 min w aplikacji ≈ +39 $, +1 min sesji w sklepie ≈ +26 $, +1 min na stronie ≈ +0,19 $ (praktycznie zero). Porównywanie tych liczb między sobą jest uprawnione tylko dlatego, że wszystkie cechy mają *std* ≈ 1 (In [4]).



In [22] prawdziwe (X) vs przewidziane (Y) wydatki — punkty na przekątnej.



In [30] histogram reszt — symetryczny, wyśrodkowany w 0.

NA CO UWAŻAĆ

- Tak dobry wynik na prawdziwych danych byłby podejrzany (np. wyciek targetu do cech). Tu wynika z tego, że dane wygenerowano z liniowego wzoru z niewielkim szumem.
- Korelacja ≠ przyczynowość: model nie mówi, że „wymuszenie" dłuższego korzystania z aplikacji podniesie wydatki.
- scikit-learn nie podaje istotności współczynników (p-value). Na pytanie „czy 0,19 dla strony www różni się od zera?" odpowie `statsmodels` (OLS → `summary()`).

WNIOSEK

Dane dobre → trening bez problemów → wynik testu bardzo dobry i wiarygodny. Dla biznesu (sekcja „Schlussfolgerungen"): wydatki wyjaśniają głównie długość członkostwa i czas w aplikacji; strona www nie ma wpływu. Czy rozwijać aplikację, czy naprawiać stronę — tego same dane nie rozstrzygają (koszty, przyczynowość). Notebook słusznie to zaznacza.

A2 Regresja logistyczna — kliknięcia w reklamę

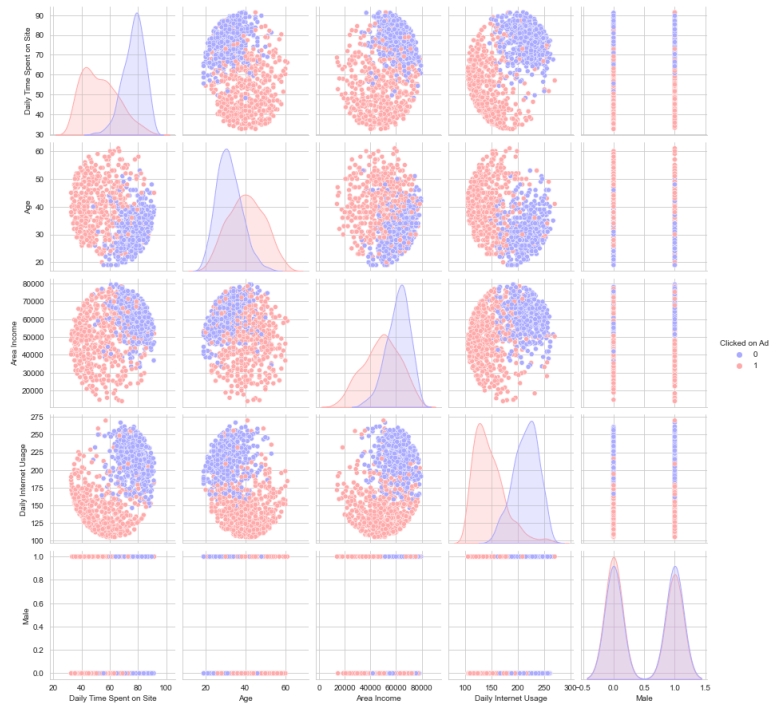
Plik	3-Logistische_Regression_Projekt-Loesung.ipynb
Zadanie	przewidzieć, czy użytkownik kliknie reklamę (`Clicked on Ad` : 1 = tak, 0 = nie)
Model	`LogisticRegression`
Typ	klasyfikacja binarna
Dane	1000 użytkowników, 10 kolumn (dane fikcyjne — „Fake-Daten")

W SKRÓCIE

✓ **Dane**	Bardzo dobre: kompletne, klasy idealnie zbalansowane (500/500), cechy wyraźnie rozdzielają klasy.
✓ **Trening**	Poprawny podział 67/33 (`random_state=42`). Drobna uwaga: brak skalowania cech o bardzo różnych zakresach.
✓ **Wynik testu**	Accuracy 0,91 przy baseline 0,51. Błędy asymetryczne: 25 przeoczonych klikających vs 6 fałszywych alarmów obliczone.

1) Dane i trening — gdzie patrzeć

In [4] `ad_data.info()`	**Widać:** 1000 wierszy, wszędzie „1000 non-null"; 4 kolumny `object` (Ad Topic Line, City, Country, Timestamp). → **Wniosek:** brak braków. Kolumny tekstowe mają setki unikalnych wartości — w In [17] świadomie pominięte (dla prostego modelu słusznie; z Timestamp można by wyciągnąć np. godzinę dnia).
In [5] `ad_data.describe()`	**Widać:** Clicked on Ad: *mean* 0,50 → dokładnie 500 × 0 i 500 × 1. Średnie „podejrzanie okrągłe" (65,0002; 55 000,00008; 180,0001). Age 19–61, Area Income ok. 14–79 tys. → **Wniosek:** klasy zbalansowane → accuracy będzie uczciwą miarą. Dane są wygenerowane. Skale bardzo różne (dochód ~55 000, wiek ~36) — patrz „Na co uważać".
In [7] `ad_data['Age'].hist(bins=30)`	**Widać:** wiek od 19 do 61 lat, większość 25–45. → **Wniosek:** sensowny rozkład, brak wartości niemożliwych.
In [15] `sns.pairplot(ad_data, hue='Clicked on Ad')`	**Widać:** (najważniejszy wykres) Daily Time Spent on Site i Daily Internet Usage tworzą dwie wyraźnie rozdzielone chmury; na przekątnej dwa przesunięte „garby". Klikający są starsi (Age) i z regionów o niższym dochodzie (Area Income). Male: rozkłady identyczne. → **Wniosek:** jeszcze przed treningiem widać, że zadanie jest łatwe — można oczekiwać accuracy ok. 90%. Cecha Male nie niesie informacji.
In [13] `jointplot(x='Daily Time Spent on Site', y='Daily Internet Usage')`	**Widać:** dwa skupiska punktów. → **Wniosek:** to te same dwie grupy (klikający / nieklikający) — dobra separowalność.
In [18] `train_test_split(X, y, test_size=0.33, random_state=42)`	**Widać:** zbiór testowy 330 przykładów (support 162 + 168 w In [23]). → **Wniosek:** proporcja klas w teście nadal ok. 50/50; wynik powtarzalny.
In [20] `LogisticRegression().fit(X_train, y_train)`	**Widać:** domyślne ustawienia, bez skalowania; brak ostrzeżeń w outpucie. → **Wniosek:** trening zakończony. Model liniowy z 6 parametrami na 670 przykładach — niskie ryzyko przeuczenia.



In [15] pairplot z `hue='Clicked on Ad'` — Daily Time Spent on Site i Daily Internet Usage rozdzielają klasy; Male (ostatnia kolumna) nie rozdziela.

2) Wyniki na danych testowych — jak czytać

In [23]

```
print(classification_report(y_test, predictions))
```

Widać: klasa 0: precision 0,86, recall 0,96; klasa 1: precision 0,96, recall 0,85; accuracy 0,91; macro ≈ weighted ≈ 0,91; support 162 / 168. → **Jak czytać:** klasa 1 (kliknął): gdy model mówi „kliknie", ma rację w 96% przypadków (precision), ale wykrywa 85% faktycznie klikających (recall) — co siódmego przeocza. Klasa 0: prawie wszystkich nieklikających rozpoznaje (96%). Macro ≈ weighted, bo klasy są równe.

baseline

Widać: największa klasa w teście: 168/330 = 50,9% obliczone. → **Wniosek:** model ma 90,6% — ok. 40 punktów procentowych ponad „zgadywanie".

Macierz pomyłek odtworzona z raportu obliczone

	przewidziane 0	przewidziane 1
prawda: 0 (nie kliknął)	156	6
prawda: 1 (kliknął)	25	143

Notebook jej nie drukuje, ale z precision, recall i support wynika jednoznacznie. Sprawdź: `confusion_matrix(y_test, predictions)`.

Accuracy
90,6%
299 z 330

Baseline
50,9%
obliczone

Recall „klik"
0,85
25 przeoczeń

Precision „klik"
0,96
6 fałszywych alarmów

macierz

Widać: 6 fałszywych alarmów (przewidziano kliknięcie, a go nie było) vs 25 przeoczeń (kliknął, a model mówi „nie"). → **Wniosek:** błędy są asymetryczne — model jest „ostrożny" w przewidywaniu kliknięcia.

wnioski biznesowe

Widać: precision 0,96 i recall 0,85 dla klasy 1. → **Wniosek:** jeśli reklama ma trafiać tylko do tych, którzy klikną (oszczędny budżet) — wysoka precision to zaleta. Jeśli nie wolno przegapić potencjalnych klientów — recall do poprawy: można obniżyć próg (`predict_proba(X_test)[:,1] > 0.4` zamiast domyślnego 0,5). Przy 330 przykładach przedział niepewności accuracy to ok. ±3 p.p. (0,87–0,94) obliczone.

NA CO UWAŻAĆ

- Brak skalowania: Area Income (~55 000) i Age (~36) w jednym modelu. Regularyzacja L2 (domyślnie `C=1`) działa wtedy nierówno na cechy, a solver może wolniej zbiegać. Lepiej: `make_pipeline(StandardScaler(), LogisticRegression())`.
- Notebook nie pokazuje macierzy pomyłek ani współczynników (`logmodel.coef_`) — a to one mówią, gdzie model się myli i które cechy „pchają" w stronę kliknięcia.

WNIOSEK

Dane dobre i dobrze rozdzielone → trening poprawny → wynik testu bardzo dobry i wiarygodny (klasy zbalansowane, więc accuracy nie oszukuje). Jedyna słabość: co siódmy klikający zostaje przeoczony.

A3 KNN — dane zanonimizowane

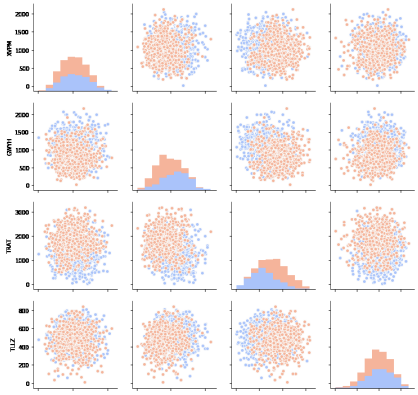
Plik	3-K_Nearest_Neighbors_Projekt-Loesung.ipynb
Zadanie	przewidzieć `TARGET CLASS` (0/1) z 10 zanonimizowanych cech
Model	`KNeighborsClassifier`; wybór K metodą „łokcia"
Typ	klasyfikacja binarna
Dane	ok. 1000 wierszy (wniosek z 300 = 30% testu), dane sztuczne

W SKRÓCIE

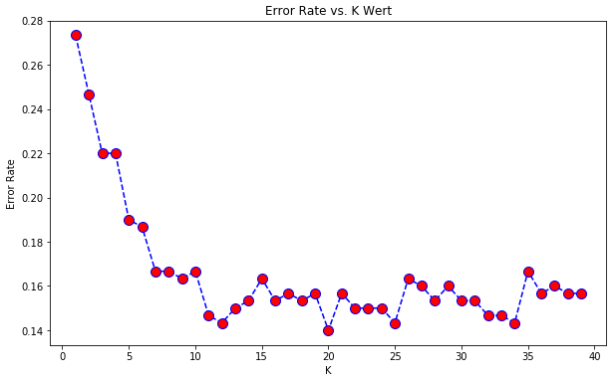
! **Dane**	Prawdopodobnie poprawne, ale niesprawdzone (brak `info()` / `describe()`). Klasy zbalansowane, jednak mocno nakładają się w każdej parze cech.
! **Trening**	Skalowanie zrobione (przy KNN konieczne), ale: scaler dopasowany na całych danych, podział bez `random_state`, K wybierane na zbiorze testowym.
✓ **Wynik testu**	K=1: 72,7%, K=24: 85,0% przy baseline 51,3%. Dobry wynik, ale lekko optymistyczny.

1) Dane i trening — gdzie patrzeć

`In [3]` `df.head()`	**Widać:** kolumny XVPM, GWYH, TRAT… (sztuczne nazwy) i bardzo różne zakresy (np. TLLZ ok. 280–725, TRAT ok. 610–2640 w pierwszych wierszach). → **Wniosek:** cechy mają różne skale, a KNN liczy odległości — skalowanie jest obowiązkowe.
brak `df.info() / df.describe()`	**Widać:** notebook od razu przechodzi do wykresu. → **Wniosek:** z notebooka nie wiemy, czy są braki lub wartości odstające. To luka w EDA — zawsze to sprawdzaj.
`In [4]` `sns.pairplot(df, hue='TARGET CLASS')`	**Widać:** w każdym wykresie 2D chmury obu klas w dużej mierze się nakładają; rozkłady na przekątnej są tylko lekko przesunięte (najbardziej np. TRAT, GWYH). → **Wniosek:** żadna pojedyncza cecha ani para cech nie rozdziela klas — informacja jest „rozproszona" po wielu wymiarach. Należy oczekiwać średniego wyniku i dużego wpływu wyboru K. Wysokości słupków na przekątnej nie mówią o liczności klas (histogramy narysowane razem); zbalansowanie potwierdza dopiero support 154/146 w In [30].
`In [7] → In [9]` `scaler.fit(df.drop('TARGET CLASS', axis=1))`	**Widać:** po skalowaniu (In [9] `df_feat.head()`) wartości leżą wokół 0 (ok. −1,8…2,2). → **Wniosek:** skale wyrównane — dobrze. Ale błąd metodyczny: scaler dopasowano na CAŁYCH danych przed podziałem (średnia i std „widziały" zbiór testowy). Przy 1000 wierszach efekt minimalny; poprawnie: najpierw podział, potem `scaler.fit(X_train)` i `transform` na obu zbiorach (tak robi notebook Keras).
`In [23]` `train_test_split(scaled_features, …, test_size=0.30)`	**Widać:** brak `random_state`. → **Wniosek:** każde uruchomienie = inny podział i inne liczby. Po ponownym uruchomieniu zobaczysz inne wyniki niż w notebooku — to normalne, nie błąd.
`In [25] / In [26]` `KNeighborsClassifier(n_neighbors=1).fit(…)`	**Widać:** KNN nie uczy się parametrów — zapamiętuje punkty treningowe. → **Wniosek:** przy K=1 accuracy na treningu wyniosłaby 100% (każdy punkt jest swoim najbliższym sąsiadem). Podręcznikowe przeuczenie — prawdę pokaże dopiero test.



In [4] fragment pairplotu (cechy XVPM, GWYH, TRAT, TLLZ) — chmury klas się nakładają.



In [32] błąd na teście vs K — stromy spadek do K≈11, potem płaskie „plateau".

2) Wyniki na danych testowych — jak czytać

In [29] — K = 1

	przew. 0	przew. 1
prawda: 0	**113**	41
prawda: 1	41	**105**

In [33] — K = 24

	przew. 0	przew. 1
prawda: 0	**131**	23
prawda: 1	22	**124**

`In [29] / In [30]` `K = 1`	**Widać:** 41 + 41 błędów na 300 → accuracy 72,7% obliczone; precision i recall obu klas ok. 0,72–0,73 (błędy symetryczne). → **Wniosek:** na treningu byłoby 100%, na teście 73% → przeuczenie: model reaguje na szum pojedynczych sąsiadów.
`In [31] / In [32]` `wykres „Error Rate vs. K Wert"`	**Widać:** błąd spada z ok. 0,27 (K=1) do ok. 0,15 dla K ≥ 11, potem „plateau" 0,14–0,165 z poszarpanymi skokami. → **Jak czytać:** wybieramy K z płaskiego obszaru, a nie z pojedynczego „dołka". Różnica 0,14 vs 0,16 to ok. 6 przykładów z 300 — szum. K=20 (minimum ok. 0,14) nie jest naprawdę lepsze od K=24 (0,15).
`In [31]` `pred_i = knn.predict(X_test)`	**Widać:** pętla liczy błąd na `X_test` / `y_test`, czyli test służy do wyboru K. → **Wniosek:** zbiór testowy przestał być „nietknięty" — końcowe 85% jest lekko zawyżone. Poprawnie: `GridSearchCV(KNeighborsClassifier(), {'n_neighbors': range(1, 40)}, cv=5)` na zbiorze treningowym, a test tylko raz, na końcu.
`In [33]` `K = 24`	**Widać:** 255/300 = 85,0% obliczone; precision i recall obu klas ok. 0,85; błędy prawie symetryczne (23 vs 22). → **Wniosek:** model nie faworyzuje żadnej klasy. Baseline 154/300 = 51,3% obliczone → model +34 p.p.; niepewność ok. ±4 p.p.
`In [30] / In [33]` `wiersz „avg / total"`	**Widać:** inny układ raportu niż w pozostałych notebookach. → **Wniosek:** to format starszej wersji scikit-learn; dziś zobaczysz *accuracy* / *macro avg* / *weighted avg*.

K = 1	K = 24	Baseline	Niepewność
72,7%	**85,0%**	**51,3%**	**±4 p.p.**
przeuczenie	255 z 300	obliczone	obliczone, n = 300

NA CO UWAŻAĆ

- Trzy drobne błędy metodyczne naraz: scaler przed podziałem, brak `random_state`, wybór K na teście. Każdy z osobna jest mały; razem dają wynik trochę optymistyczny i niepowtarzalny.
- Po ponownym uruchomieniu spodziewaj się innych liczb (dla K z plateau rzędu 82–88%) — to efekt losowego podziału.

WNIOSEK

Dane trudniejsze (klasy się nakładają), ale po skalowaniu i doborze K model działa przyzwoicie: 85% przy 51% baseline. K=1 to podręcznikowy przykład przeuczenia, a wykres błędu od K — podręcznikowy sposób, żeby to wykryć.

A4 Drzewo decyzyjne i las losowy — LendingClub

Plik	3-Decision_Trees_und_Random_Forests_Projekt-Loesung.ipynb
Zadanie	przewidzieć, czy kredyt NIE zostanie w pełni spłacony (`not.fully.paid` = 1)
Modele	`DecisionTreeClassifier()` (bez ograniczeń) i `RandomForestClassifier(n_estimators=600)`
Typ	klasyfikacja binarna, klasy niezbalansowane
Dane	9 578 kredytów LendingClub 2007–2010; 13 cech + target

W SKRÓCIE

! **Dane**	Technicznie czyste (zero braków), ale trudne: tylko 16% kredytów niespłaconych, a cechy słabo je odróżniają (EDA).
! **Trening**	Drzewo bez ograniczeń → przeuczenie. Las losowy w praktyce nauczył się mówić „spłaci”.
✗ **Wynik testu**	Oba modele bezużyteczne do wykrywania ryzyka: drzewo 73% (poniżej baseline 84,6%), las 84,55% = baseline i wykrywa 2% złych kredytów.

1) Dane i trening — gdzie patrzeć

In [3] `loans.info()`	**Widać:** 9 578 wierszy, 14 kolumn, wszędzie „9578 non-null”; jedna kolumna `object`: purpose. → **Wniosek:** brak braków (dane oczyszczono wcześniej — mówi o tym wstęp). purpose trzeba zamienić na liczby: In [13] `pd.get_dummies(…, drop_first=True)`, In [14] pokazuje 19 kolumn.
In [4] `loans.describe() – kolumna not.fully.paid`	**Widać:** *mean* = 0,160. → **Wniosek:** najważniejsza liczba w notebooku: tylko 16% kredytów to klasa 1. Model „zawsze 0” ma już ok. 84% accuracy (= baseline). Accuracy będzie tu mylące — trzeba patrzeć na recall i precision klasy 1.
In [4] `loans.describe() – min/max`	**Widać:** revol.bal max 1 207 359 przy średniej 16 914; inq.last.6mths max 33; delinq.2yrs max 13; revol.util max 119. → **Wniosek:** rozkłady silnie skośne, z wartościami odstającymi (revol.util > 100% jest możliwe przy przekroczeniu limitu, ale warto sprawdzić). Dla drzew to mały problem — dzielą po progach, więc skala i outliery niewiele zmieniają; skalowanie niepotrzebne.
In [6] `hist FICO wg credit.policy`	**Widać:** wszyscy z credit.policy = 1 mają FICO od ok. 660 w górę; poniżej jest tylko credit.policy = 0. → **Wniosek:** credit.policy to w dużej mierze reguła progowa na FICO — dwie cechy niosą częściowo tę samą informację.
In [7] `hist FICO wg not.fully.paid`	**Widać:** rozkłady obu grup prawie się pokrywają; niespłacone są tylko lekko przesunięte w stronę niższego FICO. → **Wniosek:** FICO samo słabo rozróżnia klasy.
In [8] `countplot(x='purpose', hue='not.fully.paid')`	**Widać:** niebieskie słupki (niespłacone) są małe w każdej kategorii; small_business ma wyraźnie wyższy udział (ok. ¼), credit_card i major_purchase — najniższy (ok. 1/10). → **Wniosek:** cel kredytu niesie trochę informacji, ale słabej.

`In [9] / In [10]`

```
lmplot(y='int.rate',
x='fico',
col='not.fully.paid')
```

Widać: silny trend malejący (wyższe FICO → niższe oprocentowanie), ale panele dla not.fully.paid = 0 i = 1 wyglądają podobnie. → **Wniosek:** brak wyraźnego rozdzielenia. Podsumowanie EDA: sygnał słaby, klasa docelowa mała → należy oczekiwać słabych wyników.

`In [16]`

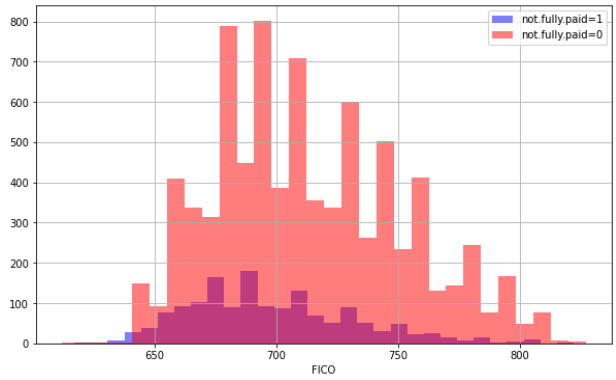
```
train_test_split(X, y,
test_size=0.30,
random_state=101)
```

Widać: test: 2 874 kredyty (support 2 431 + 443). → **Wniosek:** w teście 15,4% klasy 1 — podobnie jak w całych danych (bez `stratify`, ale proporcje i tak się zgadzają).

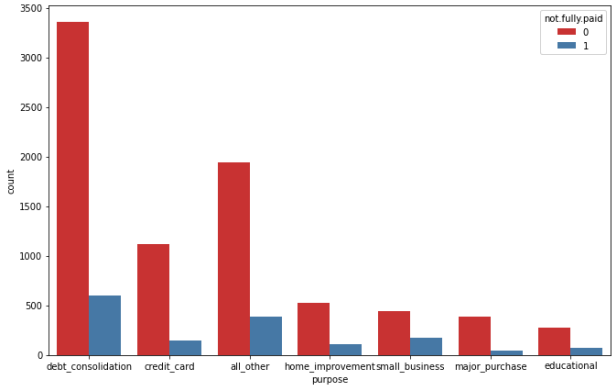
`In [18] / In [25]`

```
DecisionTreeClassifier()
/
RandomForestClassifier(n_
estimators=600)
```

Widać: domyślne ustawienia: drzewo bez limitu głębokości, brak `class_weight`, brak `random_state`. → **Wniosek:** drzewo rośnie, aż prawie każdy liść jest „czysty" → zapamiętuje dane treningowe (na treningu accuracy ≈ 100%; sprawdź `dtree.score(X_train, y_train)`). Bez `random_state` wyniki będą się trochę różnić przy każdym uruchomieniu.



In [7] FICO wg not.fully.paid — rozkłady prawie się pokrywają.



In [8] cel kredytu wg not.fully.paid — klasa 1 (niebieska) mała wszędzie.

2) Wyniki na danych testowych — jak czytać

In [23] — drzewo decyzyjne

	przew. 0	przew. 1
prawda: 0 (spłacony)	**1993**	438
prawda: 1 (niespłacony)	338	**105**

In [30] — las losowy

	przew. 0	przew. 1
prawda: 0 (spłacony)	**2421**	10
prawda: 1 (niespłacony)	434	**9**

`In [22] / In [23]`

```
drzewo decyzyjne
```

Widać: z 443 niespłaconych kredytów drzewo wykryło 105 (recall 0,24); spośród 543 alarmów „nie spłaci" trafnych było 105 (precision 0,19). → **Jak czytać:** losowe oznaczanie dałoby precision ok. 0,15 (udział klasy 1) — drzewo jest tylko ok. 1,25 raza lepsze od losowania. Accuracy 0,73 jest NIŻSZA niż baseline 0,846. Macro avg F1 0,53 ≈ poziom przypadku.

`In [29] / In [30]`

```
las losowy
```

Widać: accuracy 0,85 wygląda dobrze, ale (2 421 + 9) / 2 874 = 84,55%, a „zawsze 0" daje 2 431 / 2 874 = 84,59% obliczone. → **Jak czytać:** las jest o JEDEN przykład gorszy od modelu, który zawsze mówi „spłaci". Klasę 1 przewidział tylko 19 razy (9 trafnie): recall 0,02 → wykrywa 2% złych kredytów. Balanced accuracy 0,51 obliczone (0,50 = zgadywanie).

Baseline	Drzewo	Las losowy	Balanced acc.
84,6%	**73,0%**	**84,55%**	**0,53 / 0,51**
obliczone: „zawsze 0"	recall(1) 0,24	recall(1) 0,02	obliczone

dlaczego „0,85” myli	**Widać:** przy 85% jednej klasy accuracy mierzy głównie rozpoznawanie klasy większościowej. → **Wniosek:** zawsze patrz na wiersz klasy 1 w raporcie i na macro avg (F1 lasu: 0,48).
`In [31]` `# Kommt darauf an welche Metrik wir optimieren`	**Widać:** notebook odpowiada, że zależy od metryki i że żaden model nie był dobry. → **Wniosek:** accuracy — wygrywa las (ale = baseline); recall klasy 1, czyli wykrywanie ryzyka ważne dla inwestora — wygrywa drzewo (0,24 vs 0,02). Żaden nie nadaje się do użycia.
pytanie przed In [28] „Erhälst du etwas komisches oder eine Art Warnung?”	**Widać:** raport lasu bez ostrzeżeń. → **Wniosek:** „dziwne” jest właśnie to, że las prawie nigdy nie przewiduje klasy 1. Gdyby nie przewidział jej ani razu, sklearn wypisałby *UndefinedMetricWarning* (precision = 0/0).

NA CO UWAŻAĆ / JAK POPRAWIĆ

- `class_weight='balanced'` (drzewo i las) — błąd na klasie 1 kosztuje więcej.
- Ograniczenie drzewa: `max_depth`, `min_samples_leaf` — mniej przeuczenia.
- Zamiast `predict()`: `predict_proba()` i własny próg (np. 0,2–0,3) dobrany na krzywej precision-recall; ocena przez ROC-AUC / average precision.
- Walidacja krzyżowa (`StratifiedKFold`) zamiast jednego podziału.

WNIOSEK

Dane czyste, ale mało informacyjne i niezbalansowane → drzewo przeuczone, las zdegenerowany do „zawsze spłaci”. Wysoka accuracy lasu to iluzja — podręcznikowy przykład, dlaczego zawsze trzeba porównać wynik z baseline.

A5 SVM — irysy

Plik	3-Support_Vector_Machines_Projekt-Loesung.ipynb
Zadanie	rozpoznać gatunek irysa (setosa / versicolor / virginica) z 4 wymiarów kwiatu
Model	`SVC()` (jądro RBF) + `GridSearchCV` dla C i gamma
Typ	klasyfikacja wieloklasowa (3 klasy)
Dane	zbiór Iris: 150 kwiatów, po 50 z każdego gatunku

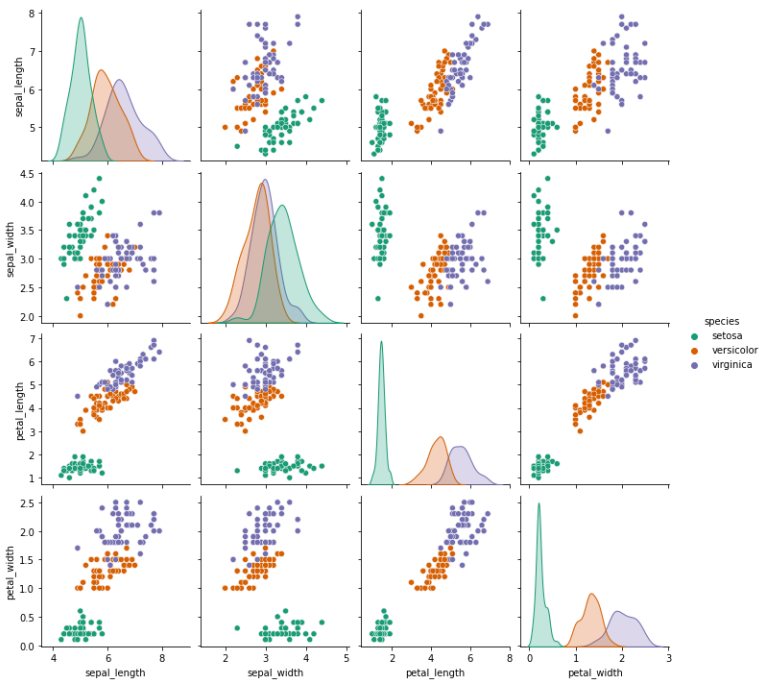
W SKRÓCIE

✓ **Dane**	Wzorcowe: kompletne, zbalansowane; setosa idealnie oddzielona, versicolor i virginica lekko się nakładają.
✓ **Trening**	Poprawny; GridSearch z 5-krotną walidacją krzyżową. Brakuje wypisania `best_params_`.
✓ **Wynik testu**	43/45 (95,6%) domyślny SVC, 42/45 (93,3%) po GridSearch — różnica jednego kwiatu to szum, nie pogorszenie.

1) Dane i trening — gdzie patrzeć

`In [4]` `sns.load_dataset('iris')`	**Widać:** notebook nie wywołuje `info()` ani `describe()`. → **Wniosek:** Iris jest znany jako czysty (150 × 5, bez braków), ale dobrą praktyką jest i tak to sprawdzić — przy innych danych byłby to błąd.

In [6] `sns.pairplot(iris, hue='species')`	**Widać:** setosa (zielona) całkowicie oddzielona w petal_length i petal_width (płatki krótsze niż 2 cm i węższe niż ok. 0,6 cm); versicolor i virginica nakładają się nieznacznie (petal_length ok. 4,5–5 cm). → **Wniosek:** setosa będzie klasyfikowana bezbłędnie; jedyne możliwe pomyłki to versicolor ↔ virginica. Tak EDA przewiduje strukturę macierzy pomyłek.
In [7] `sns.kdeplot(setosa[…], …)`	**Widać:** rozkład sepal_width vs sepal_length tylko dla setosy. → **Wniosek:** ilustracja, bez wpływu na model.
skale cech	**Widać:** wszystko w cm, od 0,1 do 7,9. → **Wniosek:** zakresy podobne, więc brak skalowania tu nie szkodzi (dla SVM zasadniczo zalecane: `StandardScaler` w Pipeline).
In [9] `train_test_split(X, y, test_size=0.30)`	**Widać:** brak `random_state` i `stratify`; w teście 11 setosa, 17 versicolor, 17 virginica (support w In [16]). → **Wniosek:** 45 kwiatów to bardzo mały test: 1 kwiat = 2,2 p.p. accuracy. Każde uruchomienie da inny podział i nieco inne wyniki.
In [19] `GridSearchCV(SVC(), param_grid, refit=True, verbose=2)`	**Widać:** „Fitting 5 folds for each of 16 candidates, totalling 80 fits"; nie wypisano `grid.best_params_` ani `grid.best_score_`. → **Wniosek:** nie wiemy, jakie C i gamma wybrano ani jaki był wynik walidacji krzyżowej — dopisz te dwie linijki. Przy 105 przykładach treningowych każdy fold walidacyjny ma ok. 21 kwiatów, więc wyniki CV „skaczą" co ok. 5 p.p. i wiele kombinacji remisuje.



In [6] pairplot z `hue='species'` — setosa wyraźnie osobno; versicolor i virginica stykają się w wymiarach płatków.

2) Wyniki na danych testowych — jak czytać

In [15] — domyślny SVC

prawda ↓ / przew. →	setosa	versic.	virgin.
setosa	**11**	0	0
versicolor	0	**15**	2
virginica	0	0	**17**

In [21] — po GridSearch

prawda ↓ / przew. →	setosa	versic.	virgin.
setosa	**11**	0	0
versicolor	0	**14**	3
virginica	0	0	**17**

`In [15] / In [16]` domyślny SVC	**Widać:** przekątna 11 + 15 + 17 = 43 z 45 (95,6%); jedyny błąd: 2 versicolor uznane za virginica. → **Jak czytać:** w raporcie to ten sam błąd widziany z dwóch stron: versicolor recall 0,88 (15/17) i virginica precision 0,89 (17/19).
`In [21] / In [22]` po GridSearch	**Widać:** 3 błędy (wszystkie versicolor → virginica), 42/45 = 93,3%. → **Wniosek:** na pytanie notebooka „Konnten wir uns verbessern?": nie — ale to też nie jest realne pogorszenie. Różnica to jeden kwiat.
niepewność	**Widać:** przy n = 45 przedział 95% dla accuracy to ok. ±6–7 p.p. obliczone. → **Wniosek:** wyniki 95,6% i 93,3% są statystycznie nie do odróżnienia. Notebook słusznie pisze, że to przede wszystkim ćwiczenie GridSearch.
jak porównywać `cross_val_score(model, X, y, cv=10)`	**Widać:** jeden podział train/test na 150 próbkach. → **Wniosek:** przy małych danych porównuj modele walidacją krzyżową (średnia ± odchylenie z 10 podziałów), a nie jednym testem.

SVC	GridSearch	Baseline	Niepewność
95,6%	**93,3%**	**37,8%**	**±6–7 p.p.**
43 z 45	42 z 45	obliczone: 17 z 45	obliczone, n = 45

NA CO UWAŻAĆ

- Mały test i brak `random_state` → wynik może się zmieniać o kilka punktów przy każdym uruchomieniu.
- GridSearch wybiera parametry na podstawie walidacji krzyżowej, a nie testu — gorszy wynik na 45 przykładach nie znaczy, że wybór był zły.

WNIOSEK

Dane idealne i dobrze rozdzielone → trening bez problemów → wynik testu bardzo dobry (93–96%). Jedyne błędy pojawiają się dokładnie tam, gdzie pairplot pokazał nakładanie się klas (versicolor/virginica).

A6 K-Means — uczelnie w USA

Plik	3-K_Means_Clustering_Projekt-Loesung.ipynb
Zadanie	bez użycia etykiet podzielić 777 uczelni na 2 klastry i sprawdzić, czy pokrywają się z podziałem prywatne/publiczne
Model	`KMeans(n_clusters=2)`
Typ	uczenie nienadzorowane (klastrowanie); etykieta Private użyta tylko do oceny
Dane	College_Data: 777 uczelni, 17 cech liczbowych

W SKRÓCIE

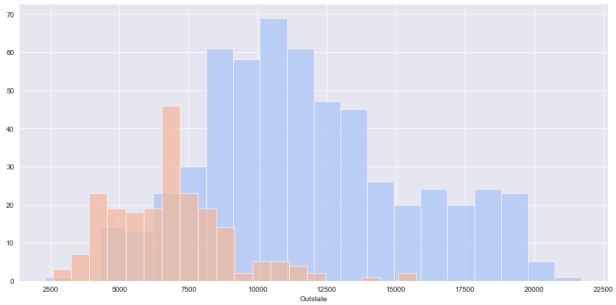
! **Dane**	Kompletne, ale z błędnymi wartościami (Grad.Rate 118%, PhD 103%) i ze skalami cech różniącymi się nawet tysiąckrotnie.
! **Trening**	K-Means na 17 nieskalowanych cechach → klastry wyznaczają cechy „wielkości uczelni", a nie status prywatna/publiczna.
✗ **Wynik**	Notebook pokazuje accuracy 0,22, ale to artefakt numeracji klastrów. Po poprawnym dopasowaniu: 0,78 przy baseline 0,73 → klastry tylko słabo odpowiadają podziałowi prywatne/publiczne.

1) Dane i trening — gdzie patrzeć

In [26] `df.info()`	**Widać:** 777 wierszy (wstęp mówi o 770 — drobna niezgodność), wszędzie non-null; Private jako `object` (Yes/No). → **Wniosek:** brak braków.
In [27] `df.describe() — wiersz max`	**Widać:** Grad.Rate max = 118 (odsetek absolwentów > 100% jest niemożliwy) i PhD max = 103 (odsetek wykładowców z doktoratem > 100%). → **Wniosek:** dwa błędy w danych. Notebook znajduje pierwszy (In [32]: Cazenovia College) i poprawia go na 100. Drugiego nie zauważa — to Texas A&M University at Galveston (sprawdziłem). Lekcja: przejrzyj min/max KAŻDEJ kolumny, zwłaszcza procentowych.
In [27] `df.describe() — wiersz std`	**Widać:** Apps ≈ 3 870, F.Undergrad ≈ 4 850, Expend ≈ 5 222, Outstate ≈ 4 023, a S.F.Ratio ≈ 4, Grad.Rate ≈ 17. → **Wniosek:** K-Means liczy odległości euklidesowe, więc cechy o dużych liczbach zdominują wynik. Notebook nie skaluje danych.
In [29] / In [30] `lmplot('Outstate', 'F.Undergrad', hue='Private'); hist Outstate`	**Widać:** wyraźny podział: uczelnie publiczne są duże (F.Undergrad do 30 tys.) i tanie dla studentów spoza stanu (Outstate zwykle < 10 tys.); prywatne — małe i droższe. In [28] (Room.Board vs Grad.Rate): duże nakładanie. → **Wniosek:** EDA pokazuje, które cechy niosą podział prywatne/publiczne: Outstate i F.Undergrad.
In [33] / In [34] `df['Grad.Rate']['Cazenovia College'] = 100`	**Widać:** ostrzeżenie *SettingWithCopyWarning*; In [34] potwierdza poprawkę (pusta tabela). → **Wniosek:** w tej, starszej wersji pandas zadziałało. W pandas ≥ 3.0 taki „łańcuchowy" zapis NIE zmienia DataFrame (sprawdziłem) — używaj `df.loc['Cazenovia College', 'Grad.Rate'] = 100`.
In [37] / In [38] `KMeans(n_clusters=2).fit(df.drop('Private', axis=1))`	**Widać:** brak skalowania i brak `random_state`. → **Wniosek:** (a) klastry zdominują cechy o dużych liczbach; (b) numeracja klastrów (który to „0", a który „1") jest losowa i może się zmieniać przy każdym uruchomieniu.



In [29] Outstate vs F.Undergrad — publiczne (pomarańczowe) duże i tańsze, prywatne (niebieskie) małe i droższe.



In [30] histogram Outstate wg Private — najlepsza pojedyncza cecha rozdzielająca.

2) Wyniki — jak czytać

In [39] `kmeans.cluster_centers_`	**Widać:** dwa wiersze po 17 liczb (notacja naukowa). Klaster 1: Apps ≈ 10 363, Accept ≈ 6 551, Enroll ≈ 2 570, F.Undergrad ≈ 13 062; klaster 0: Apps ≈ 1 813, F.Undergrad ≈ 2 189. Outstate: 10 396 vs 10 719 — prawie identycznie! → **Jak czytać:** porównuj kolumny między wierszami. K-Means podzielił uczelnie na „duże" i „małe", a nie na „drogie prywatne" i „tanie publiczne". obliczone: 97% różnicy między centrami (w metryce euklidesowej) pochodzi z F.Undergrad, Apps, Accept i Expend; udział Outstate ≈ 0%.

In [43] — tak jak w notebooku

	klaster 0	klaster 1
prawda: publiczna (0)	**138**	74
prawda: prywatna (1)	531	**34**

Te same klastry, odwrócone nazwy obliczone

	klaster 1 → „0"	klaster 0 → „1"
prawda: publiczna (0)	**74**	138
prawda: prywatna (1)	34	**531**

In [43] `confusion_matrix(df['Cluster'], kmeans.labels_)`	**Widać:** klaster 0 (669 uczelni) = 531 prywatnych + 138 publicznych; klaster 1 (108 uczelni) = 74 publiczne + 34 prywatne. → **Wniosek:** klaster 1 to po prostu duże uczelnie (głównie publiczne).

In [44]
`classification_report(…)`
`→ accuracy 0.22`
	Widać: notebook zakłada „klaster 1 = prywatna" i dostaje accuracy 0,22. → **Jak czytać:** K-Means nie wie nic o etykietach — numery 0/1 nadaje losowo. Przy odwrotnym przypisaniu (klaster 0 = prywatna, klaster 1 = publiczna): (531 + 74) / 777 = 77,9% obliczone. dodatkowy test: na tych samych danych przy 20 różnych `random_state` dokładnie ta macierz wyszła 13 razy, a odwrócona — 7 razy. Ten sam model może więc pokazać „22%" albo „78%".

właściwa ocena	**Widać:** najlepsze dopasowanie 77,9% vs baseline „zawsze prywatna" 565/777 = 72,7% obliczone. → **Wniosek:** tylko +5 p.p. ponad trywialną odpowiedź. Uczelnie publiczne rozpoznane w 35% (74 z 212). Adjusted Rand Index (miara niezależna od numeracji: 0 = losowo, 1 = idealnie) = 0,23 → słaba zgodność.
In [44] „Gar nicht so schlecht…"	**Widać:** komentarz notebooka chwali wynik. → **Wniosek:** 22% to nie „całkiem nieźle", tylko odwrócone etykiety; po odwróceniu wynik jest ledwie lepszy od baseline. Ten komentarz jest mylący.

Notebook (In [44])	Po dopasowaniu	Baseline	ARI
22,1%	**77,9%**	**72,7%**	**0,23**
etykiety odwrócone	obliczone	„zawsze prywatna"	obliczone

DODATKOWY TEST (POZA NOTEBOOKIEM, TE SAME DANE)

- **Wszystkie 17 cech po `StandardScaler`:** K-Means dzieli uczelnie na „elitarne/selektywne" i pozostałe (największe różnice centrów: Top10perc, Top25perc, PhD, Terminal, Outstate). Zgodność z podziałem prywatne/publiczne jest jeszcze gorsza: 52%, ARI ≈ 0.
- **Tylko 2 cechy wskazane przez EDA** (Outstate + F.Undergrad, skalowane): 87,4% zgodności, ARI 0,52.
- **Lekcja:** K-Means znajduje dominującą strukturę w podanych mu cechach, a nie „tę, o którą nam chodzi". O wyniku decydują wybór cech i skalowanie.

NA CO UWAŻAĆ

- W prawdziwym zastosowaniu (bez etykiet) nie ma macierzy pomyłek — notebook sam zaznacza, że to tylko „teoretyczna" kontrola. Jakość klastrów ocenia się wtedy np. przez *silhouette score*, metodę łokcia (inertia vs k) i interpretację centrów.
- Porównując klastry z etykietami, używaj miar niezależnych od numeracji (`adjusted_rand_score`) albo najlepszego dopasowania etykiet.

WNIOSEK

Dane wymagały czyszczenia (2 niemożliwe wartości) oraz skalowania i wyboru cech. Klastry odzwierciedlają wielkość uczelni, nie jej status. Wynik 0,22 to artefakt numeracji; realnie 0,78 przy baseline 0,73 — słabo.

A7 NLP — recenzje Yelp

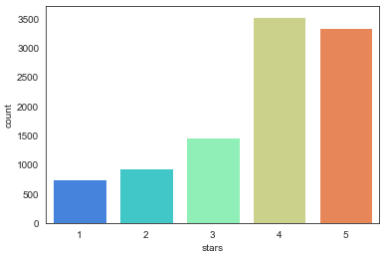
Plik	3-NIp_Projekt-Loesung.ipynb
Zadanie	na podstawie tekstu recenzji rozpoznać 1 gwiazdkę vs 5 gwiazdek
Modele	(1) `CountVectorizer` + `MultinomialNB`; (2) Pipeline: `CountVectorizer` + `TfidfTransformer` + `MultinomialNB`
Typ	klasyfikacja tekstu (binarna), klasy niezbalansowane
Dane	10 000 recenzji Yelp; do zadania 4 086 (749 × 1★, 3 337 × 5★)

W SKRÓCIE

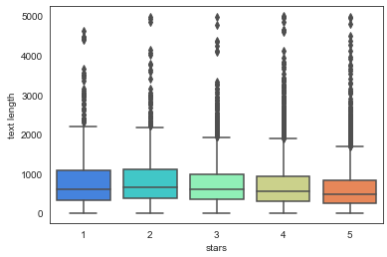
! **Dane**	Kompletne (zero braków), ale niezbalansowane: tylko 18% recenzji ma 1★.
! **Trening**	Model 1 poprawny (z drobnym przeciekiem: słownik budowany na całych danych). Model 2 (TF-IDF) zdegenerowany.
! **Wynik testu**	Model 1: 92,6% przy baseline 81,4%, ale przeocza 30% recenzji 1★. Model 2: zawsze „5★" = dokładnie baseline — bezużyteczny, a raport w notebooku to ukrywa.

1) Dane i trening — gdzie patrzeć

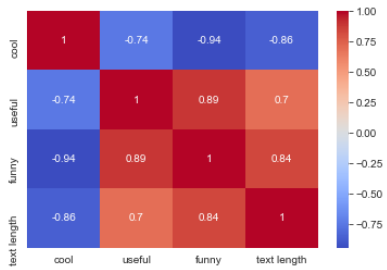
`In [4]` `yelp.info()`	**Widać:** 10 000 wierszy, wszędzie non-null; text jako `object`. → **Wniosek:** brak braków.
`In [5]` `yelp.describe()`	**Widać:** stars średnio 3,78; cool/useful/funny — max 77/76/57 przy medianie 0–1. → **Wniosek:** oceny przesunięte w stronę pozytywnych; głosy silnie skośne (w modelu i tak nieużywane).
`In [8] / In [9]` `FacetGrid(col='stars') / boxplot text length`	**Widać:** kształty histogramów podobne, mediany długości tekstu prawie równe dla 1–5★, dużo wartości odstających. → **Wniosek:** długość tekstu słabo różnicuje oceny — nie byłaby dobrą cechą. (Różne wysokości słupków w FacetGrid to po prostu różne liczności klas.)
`In [10]` `sns.countplot(x='stars')`	**Widać:** 4★ i 5★ dominują (ok. 3 500 i 3 300), 1★ najmniej (ok. 750). → **Wniosek:** w zadaniu 1★ vs 5★ klasy są niezbalansowane: 749 vs 3 337, czyli 18,3% to 1★ (obliczone). Baseline „zawsze 5★" ≈ 81%.
`In [11] → In [13]` `groupby('stars').mean() → corr() → heatmap`	**Widać:** korelacje rzędu −0,94 i 0,89. → **Wniosek:** uwaga: liczone na 5 wierszach (średnie dla 1…5 gwiazdek), a nie na 10 000 recenzjach. Przy n = 5 tak wysokie korelacje mogą wyjść przypadkiem; to korelacja średnich grup, a nie zależność między pojedynczymi recenzjami (błąd ekologiczny). Nie wyciągaj z tej heatmapy wniosków o modelu.
`In [14]` `yelp[(yelp.stars==1) | (yelp.stars==5)]`	**Widać:** zostają tylko 1★ i 5★. → **Wniosek:** 4 086 recenzji do klasyfikacji.
`In [17] → In [19]` `X = cv.fit_transform(X), potem train_test_split`	**Widać:** słownik (dodatkowy test: 19 183 słowa) zbudowano także z recenzji testowych. → **Wniosek:** drobny przeciek. Poprawnie: najpierw podział, potem `fit` tylko na treningu (tak działa Pipeline w drugiej części). dodatkowy test: przy poprawnej kolejności wynik jest prawie identyczny ([[155, 73], [15, 983]], 92,8%) — tu przeciek nic nie zmienił, ale nawyk jest ważny.
`In [19]` `train_test_split(X, y, test_size=0.3, random_state=101)`	**Widać:** test = 1 226 recenzji (228 × 1★, 998 × 5★ — support w In [24]). → **Wniosek:** proporcje jak w całości; wynik powtarzalny.



In [10] liczba recenzji wg gwiazdek — 1★ to najmniejsza grupa.



In [9] długość tekstu wg gwiazdek — mediany prawie równe.



In [13] heatmapa korelacji — liczona tylko na 5 średnich.

2) Wyniki na danych testowych — jak czytać

In [24] — model 1: Count + NB

	przew. 1★	przew. 5★
prawda: 1★	**159**	69
prawda: 5★	22	**976**

In [31] — model 2: TF-IDF + NB

	przew. 1★	przew. 5★
prawda: 1★	**0**	228
prawda: 5★	**0**	**998**

Kolumna samych zer: model nigdy nie wybiera 1★.

`In [24]`
`model 1 — macierz pomyłek`

Widać: przekątna 159 + 976 = 1 135 trafień z 1 226 = 92,6%. 22 recenzje 5★ uznane za 1★; 69 recenzji 1★ uznanych za 5★. → **Jak czytać:** wiersze = prawda, kolumny = przewidywanie, kolejność klas 1★, 5★.

`In [24]`
`model 1 — classification_report`

Widać: 1★: precision 0,88, recall 0,70; 5★: precision 0,93, recall 0,98; accuracy 0,93; macro avg F1 0,87, weighted 0,92. → **Jak czytać:** gdy model mówi „1★", ma rację w 88% (159 ze 181). Ale z 228 prawdziwych negatywnych recenzji wykrywa 159 — co trzecią przeocza. Jeśli celem jest wyłapywanie niezadowolonych klientów, to właśnie recall 1★ jest kluczowy. Różnica macro (0,87) vs weighted (0,92) zdradza słabszą małą klasę.

baseline

Widać: 998 / 1 226 = 81,4% (obliczone). → **Wniosek:** model 1 jest o 11 p.p. lepszy — naprawdę się nauczył. Balanced accuracy 0,84 (obliczone).

Model 1	Baseline	Recall 1★	Model 2
92,6%	**81,4%**	**0,70**	**81,4%**
accuracy	obliczone	69 przeoczeń	= baseline, recall 1★ = 0

`In [31]`
`model 2 — macierz [[0, 228], [0, 998]]`

Widać: pierwsza kolumna to same zera. → **Jak czytać:** model NIGDY nie przewiduje 1★. To klasyfikator stały: accuracy = 998 / 1 226 = 81,4% = dokładnie baseline; balanced accuracy 0,50 (obliczone) = poziom zgadywania.

`In [31]`
`classification_report(…, labels=np.unique(predictions))`

Widać: tylko wiersz klasy 5 (precision 0,81, recall 1,00, f1 0,90) i dziwny wiersz „micro avg". → **Wniosek:** raport maskuje problem! Parametr `labels=np.unique(predictions)` ogranicza raport do klas, które model przewidział — a przewidział tylko 5★. Pełny raport (bez `labels`) pokazałby dla 1★ precision 0 i recall 0 (z ostrzeżeniem *UndefinedMetricWarning*). Zasada: najpierw macierz pomyłek.

DLACZEGO TF-IDF „ZEPSUŁ" MODEL? (DODATKOWY TEST, TE SAME DANE)

Notebook tylko stwierdza, że TF-IDF pogorszył wynik. Przyczyna leży w połączeniu TF-IDF z `MultinomialNB` i wygładzaniem `alpha=1`:

- Wagi TF-IDF to małe ułamki: na recenzję przypada ich łącznie ok. 7, zamiast ok. 113 zliczeń słów.
- Łączna „masa" TF-IDF wszystkich recenzji 1★ w treningu to ok. 4 100, a wygładzanie Laplace'a dodaje 1 do każdego z 16 233 słów słownika — jest więc ok. 4× większe niż dane tej klasy.
- Rozkład słów klasy 1★ zostaje „rozmyty" prawie do równomiernego. Wtedy nawet neutralne słowa przemawiają za 5★ — tak było dla 99,8% recenzji testowych.
- Dowód: `fit_prior=False` (wyłączenie wpływu proporcji klas) nie pomaga — 2 z 228 wykrytych 1★; mniejsze wygładzanie pomaga: `alpha=0.01` → 121 z 228. Najlepiej zadziałało TF-IDF + `LogisticRegression(class_weight='balanced')`: 93,4% accuracy, recall 1★ = 0,88 (201 z 228), balanced accuracy 0,91.

NA CO UWAŻAĆ

- Przy niezbalansowanych klasach zawsze porównuj accuracy z baseline (tu 81%) i patrz na recall małej klasy oraz macro avg.
- Nigdy nie ograniczaj `classification_report` parametrem `labels` do przewidzianych klas.
- Przetwarzanie tekstu (CountVectorizer, TF-IDF) dopasowuj tylko na zbiorze treningowym — Pipeline robi to automatycznie.

WNIOSEK

Dane dobre, ale niezbalansowane. Model 1 (Count + NB) naprawdę działa: 93% przy 81% baseline, choć przeocza co trzecią negatywną recenzję. Model 2 (TF-IDF + NB z `alpha=1`) to model stały — jego 81% to wyłącznie klasa większościowa.

A8 Keras — LendingClub (sieć neuronowa)

Plik	04-Keras-Projekt-Uebung-Loesung.ipynb
Zadanie	przewidzieć, czy kredytobiorca spłaci kredyt (`loan_repaid`: 1 = Fully Paid, 0 = Charged Off)
Model	Keras `Sequential`: Dense 78 → 39 → 19 → 1, Dropout 0,2, sigmoid, `binary_crossentropy`, Adam, 25 epok
Typ	klasyfikacja binarna, klasy niezbalansowane (80/20)
Dane	396 030 kredytów, 27 kolumn; model trenowany na 10% próbie (39 522 wiersze)

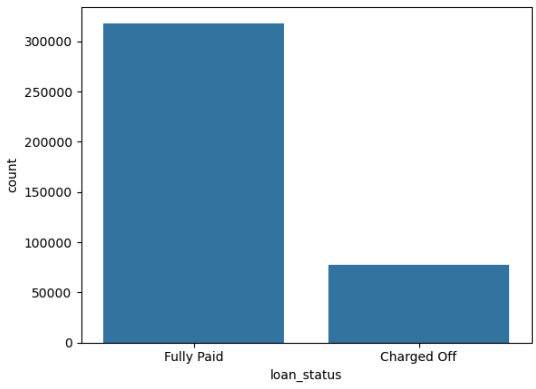
Odnośniki: plik zawiera wyniki z dwóch przebiegów, więc niektóre numery In [n] się powtarzają — kieruj się fragmentem kodu i nagłówkiem sekcji (Abschnitt 1–3).

W SKRÓCIE

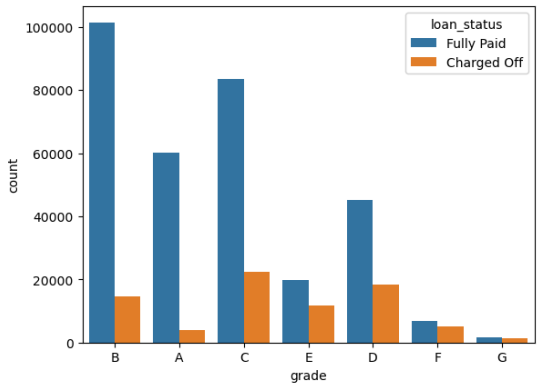
! **Dane**	Surowe dane wymagały dużo pracy: braki (do 9,5%), 15 kolumn tekstowych, cechy zdublowane i jedna „przeciekająca". Notebook obrabia to wzorowo. Klasy 80/20.
✓ **Trening**	Krzywa loss/val_loss: szybka zbieżność; od ok. 5. epoki val_loss płaski, później lekko rośnie → łagodne przeuczenie.
! **Wynik testu**	Accuracy 88,1% przy baseline 80,1% (+8 p.p.), ale recall dla „Charged Off" tylko 0,46 → model wykrywa mniej niż połowę złych kredytów.

1a) Dane — Abschnitt 1: Explorative Datenanalyse

In [8] `df.info()`	**Widać:** 396 030 wierszy, 27 kolumn; *Non-Null Count* < 396 030 w: emp_title, emp_length, title, revol_util, mort_acc, pub_rec_bankruptcies; 15 kolumn `object`. → **Wniosek:** są braki i dużo tekstu — dane wymagają przygotowania.
In [9] / In [17] `countplot(x='loan_status'); groupby('loan_status')['loan_amnt'].describe()`	**Widać:** Fully Paid 318 357 vs Charged Off 77 673 (80,4% / 19,6% obliczone). → **Wniosek:** klasy niezbalansowane; baseline „zawsze spłaci" ≈ 80%. Z tą liczbą trzeba porównać accuracy.
In [16] / In [17] `boxplot(x='loan_status', y='loan_amnt')`	**Widać:** mediana 14 000 (Charged Off) vs 12 000 (Fully Paid); pudełka mocno się nakładają. → **Wniosek:** kwota kredytu to słaby predyktor.
In [15] `scatterplot(x='installment', y='loan_amnt')`	**Widać:** prawie liniowa zależność (rata wynika z kwoty, oprocentowania i okresu). → **Wniosek:** informacja zdublowana — notebook pyta o to wprost. Dla sieci to nie błąd, ale cecha nic nowego nie wnosi.
In [20] i dalej `countplot(x='grade', hue='loan_status'); sub_grade; F i G`	**Widać:** udział Charged Off rośnie systematycznie od A do G; w F i G jest prawie 50/50. → **Wniosek:** grade/sub_grade to silny predyktor (w praktyce to ocena ryzyka LendingClub).
In [27] `df.corr()['loan_repaid'].sort_values().drop('loan_repaid').plot(kind='bar')`	**Widać:** najsilniejsza int_rate ≈ −0,25; pozostałe |r| < 0,1. → **Wniosek:** pojedyncze cechy liczbowe są słabe — potrzebna kombinacja wielu cech (to uzasadnia sieć neuronową).
In [11] / In [12] `df.corr() → ValueError: could not convert string to float: ' 36 months'`	**Widać:** błąd zamiast heatmapy. → **Wniosek:** to błąd z nowszego przebiegu, nie z danych: od pandas 2.0 `corr()` nie pomija kolumn tekstowych. Naprawa: `df.corr(numeric_only=True)`.



In [9] loan_status — ok. 80% Fully Paid: baseline ≈ 80%.



In [20] grade wg loan_status — udział Charged Off rośnie od A do G.

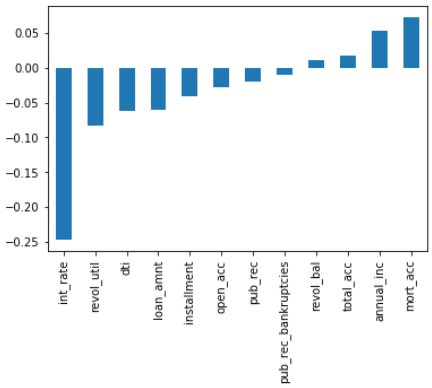
1b) Dane — Abschnitt 2: Daten-Preprocessing

In [30] / In [31] `df.isnull().sum() / w %`	**Widać:** mort_acc 9,5%, emp_title 5,8%, emp_length 4,6%, title 0,4%, pub_rec_bankruptcies 0,14%, revol_util 0,07%. → **Wniosek:** decyzje notebooka: emp_title usunięty (In [33]: 173 105 unikalnych wartości — nie do zakodowania); emp_length usunięty (patrz niżej); title usunięty (In [48]/[49]: opisuje to samo co purpose); mort_acc uzupełniony średnią w grupach total_acc (In [53]: najwyższa korelacja 0,38) — rozsądne; revol_util i pub_rec_bankruptcies: `dropna()` (< 0,5% wierszy).

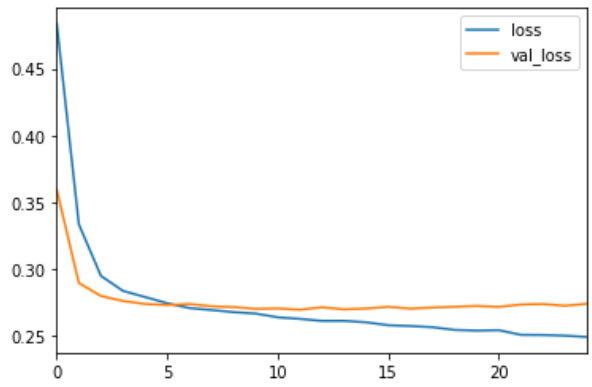
`In [44] / In [45]` `emp_len = emp_co / emp_fp`	**Widać:** wartości 0,23–0,26 w każdej kategorii stażu pracy. → **Jak czytać:** to stosunek Charged Off do Fully Paid (ile niespłaconych na jeden spłacony), a nie procent — choć zadanie mówi o „Prozentanteil". Udział niespłaconych = r/(1+r) ≈ 18–21% obliczone we wszystkich grupach; płaski profil = cecha bez informacji, słusznie usunięta.
`In [64] → In [77]` `kodowanie kategorii`	**Widać:** term → 36/60; grade usunięty (zawarty w sub_grade); dummies: sub_grade, verification_status, application_type, initial_list_status, purpose, home_ownership (NONE/ANY → OTHER); zip_code z address → dummies; earliest_cr_line → rok. → **Wniosek:** na końcu tekstem jest już tylko loan_status (In [77]), usunięty potem jako duplikat loan_repaid. Model dostaje 78 cech — zgadza się z `Dense(78)`.
`In [75]` `df.drop('issue_d', axis=1)`	**Widać:** uzasadnienie: „Datenüberlauf" (wyciek danych). → **Wniosek:** wzorcowe myślenie: przy decyzji o kredycie nie znamy daty jego wypłaty — cecha byłaby dostępna dopiero „po fakcie".
`In [28]` `df.head() – kolumna address`	**Widać:** „Mendozaberg, OK 22690", „Loganmouth, SD 05113" i „New Sabrina, WV 05113" — ten sam kod pocztowy w dwóch stanach. → **Wniosek:** adresy są syntetyczne. Ewentualna siła predykcyjna zip_code to artefakt tego kursowego zbioru, a nie prawidłowość ze świata. Sprawdź: `df.groupby('zip_code')['loan_repaid'].mean()`.
`In [82]` `print(len(df)) → 39522`	**Widać:** linia `df.sample(frac=0.1, …)` jest teraz zakomentowana, ale wynik 39 522 pokazuje, że w przebiegu, z którego pochodzą wyniki, użyto 10% próby. → **Wniosek:** 395 219 × 0,1 ≈ 39 522; test 20% = 7 905 — zgadza się z support w raporcie obliczone.
`In [88] / In [89]` `scaler.fit_transform(X_train); scaler.transform(X_test)`	**Widać:** MinMaxScaler dopasowany tylko na treningu. → **Wniosek:** poprawnie, bez przecieku (notebook wprost to uzasadnia).

1c) Trening — jak przebiegł

`In [24]` `Dense(78) … Dense(1, activation='sigmoid'); compile(loss='binary_crossentropy')`	**Widać:** architektura lejkowa z Dropout 0,2 po każdej warstwie ukrytej. → **Wniosek:** standard dla klasyfikacji binarnej; Dropout ogranicza przeuczenie.
`In [25]` `model.fit(…, epochs=25, batch_size=256, validation_data=(X_test, y_test))`	**Widać:** zbiór testowy służy też jako walidacyjny. → **Wniosek:** przy obserwowaniu krzywych (a tym bardziej przy early stopping) test przestaje być niezależny. Poprawnie: osobny zbiór walidacyjny (np. `validation_split=0.1`), test — tylko raz, na końcu.
`In [97]` `losses[['loss','val_loss']].plot()`	**Widać:** loss spada stale z ok. 0,49 do ok. 0,25; val_loss spada szybko do ok. 0,27 (ok. 5 epok), potem jest płaski i od ok. 15. epoki lekko rośnie (do ok. 0,274), choć loss dalej spada. → **Jak czytać:** rosnąca luka między krzywymi = początek przeuczenia (sieć uczy się szczegółów zbioru treningowego, które nie uogólniają się). Na początku val_loss < loss — to normalne: Dropout działa tylko w treningu, a loss to średnia z całej epoki. Wystarczyłoby ok. 5–10 epok; `EarlyStopping(monitor='val_loss', patience=3)` zatrzymałby trening automatycznie. Przeuczenie jest łagodne (luka ok. 0,025), więc wyniki pozostają wiarygodne.



In [27] korelacja cech z loan_repaid — tylko int_rate wyraźnie (≈ −0,25).



In [97] loss (trening) vs val_loss — val_loss płaski od ok. 5. epoki, luka rośnie.

2) Wyniki na danych testowych — jak czytać

In [101] — confusion_matrix(y_test, predictions)

	przew. 0	przew. 1
prawda: 0 Charged Off	**723**	847
prawda: 1 Fully Paid	95	**6240**

847 = złe kredyty uznane za dobre (najdroższy błąd dla pożyczkodawcy).

Accuracy
88,1%
6 963 z 7 905

Baseline
80,1%
obliczone

Recall Charged Off
0,46
847 przeoczonych

Balanced acc.
0,72
obliczone

`In [101]` `kodowanie klas`	**Widać:** klasa 0 = Charged Off (kredyt niespłacony), 1 = Fully Paid; wiersze = prawda, kolumny = przewidywanie. → **Wniosek:** uważaj: tu „klasa 0" jest tą ważniejszą (ryzyko).
`In [100]` `classification_report(y_test, predictions)`	**Widać:** klasa 0: precision 0,88, recall 0,46; klasa 1: precision 0,88, recall 0,99; accuracy 0,88; macro avg recall 0,72 / F1 0,77; weighted 0,88 / 0,87. → **Jak czytać:** model wykrywa 46% złych kredytów (723 z 1 570); gdy ostrzega „nie spłaci", ma rację w 88%. Dobre kredyty rozpoznaje prawie wszystkie (99%). Różnica macro vs weighted = słabość na mniejszej klasie.
`baseline`	**Widać:** 6 335 / 7 905 = 80,1% (obliczone). → **Wniosek:** model 88,1% = +8 p.p. Realna, ale umiarkowana poprawa.
`interpretacja biznesowa`	**Widać:** (obliczone): gdyby bank przyznawał kredyty tylko tym, którym model daje „1": z 7 087 zaakceptowanych 847 by nie spłaciło — 12,0% zamiast 19,9% bez modelu. Odrzuconych byłoby 818 wniosków, z czego 95 (11,6%) to dobrzy klienci. → **Wniosek:** w takim języku warto opisywać wynik: o ile spada odsetek złych kredytów i ilu dobrych klientów tracimy.
`In [99]` `model.predict_classes(X_test)`	**Widać:** domyślny próg 0,5. → **Wniosek:** przy niezbalansowanych klasach można go przesunąć (np. „spłaci" dopiero przy p > 0,6–0,7): więcej wykrytych złych kredytów kosztem odrzucenia większej liczby dobrych. Wybór zależy od kosztów obu błędów — pomaga krzywa precision-recall.
`In [102] → In [104]` `new_customer … predict … loan_repaid`	**Widać:** model przewiduje 1, prawda to 1. → **Wniosek:** jeden przykład niczego nie dowodzi — model „zawsze 1" też miałby tu rację (i ma w 80% przypadków). Poza tym ten klient pochodzi z df, więc z prawdopodobieństwem ok. 80% był w zbiorze treningowym — to nie jest „nowy" klient.

STAN TECHNICZNY PLIKU (WAŻNE PRZY PONOWNYM URUCHAMIANIU)

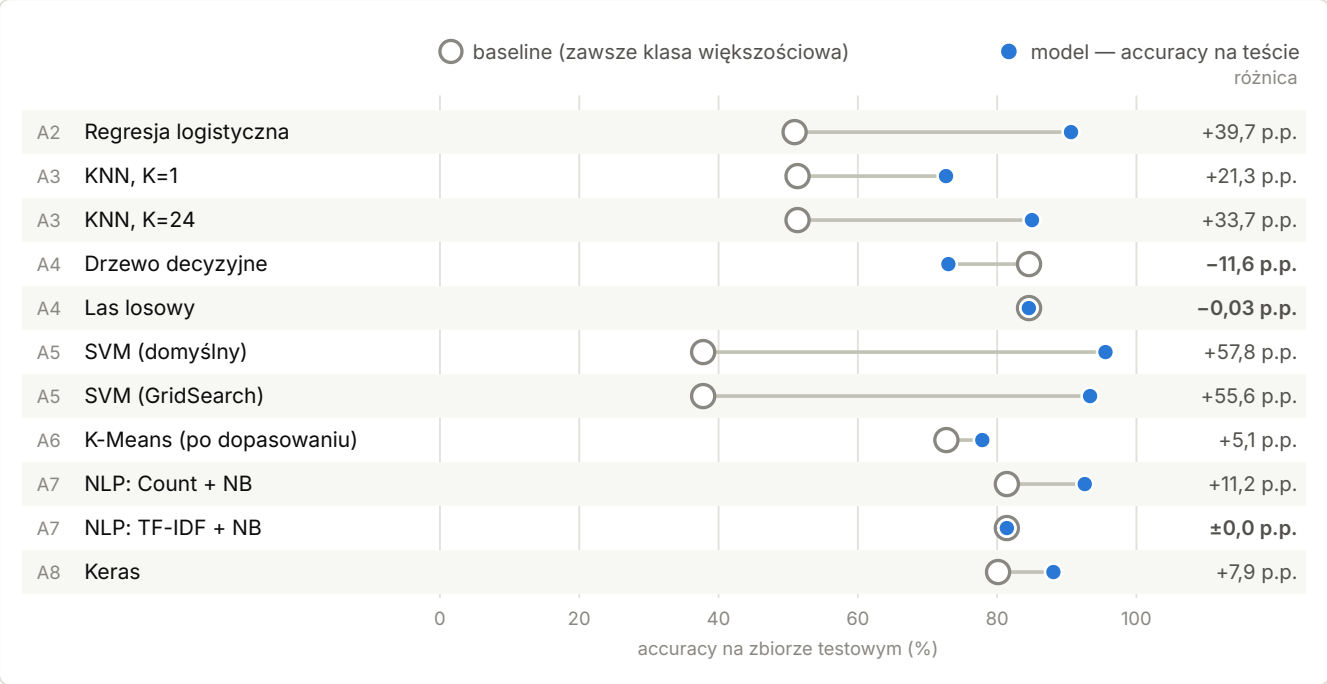
- Nowszy przebieg (14.10.2026, Python 3.13.9) obejmuje In [1]–[21] (EDA) i In [22]–[29] (komórki modelu na końcu); pozostałe wyniki — m.in. wykres loss, raport i macierz — pochodzą ze starszego przebiegu.
- In [11]/[12] `df.corr()` → ValueError: w pandas ≥ 2.0 użyj `df.corr(numeric_only=True)`.
- In [25] `fit` → NameError (X_train nie istnieje); In [26] → NameError (train_test_split zaimportowany dopiero w In [27]); In [29] → KeyError 'loan_repaid' (pominięto całe przygotowanie danych). Rozwiązanie: *Kernel → Restart & Run All*, czyli wszystko po kolei.
- `model.predict_classes` nie istnieje w nowym TensorFlow (usunięte w 2.6): `(model.predict(X_test) > 0.5).astype(int)`.
- `df.groupby('total_acc').mean()['mort_acc']` → w pandas ≥ 2.0: `df.groupby('total_acc')['mort_acc'].mean()`.

WNIOSEK

Dane wymagały dużo pracy (braki, kategorie, przeciek) — notebook robi to wzorowo. Trening przebiegł poprawnie, z łagodnym przeuczeniem od ok. 10.–15. epoki. Wynik: +8 p.p. ponad baseline, świetne rozpoznawanie dobrych kredytów, ale tylko 46% złych wykrytych. Przed praktycznym użyciem trzeba dostroić próg albo wagi klas.

A9 Porównanie wszystkich modeli i najważniejsze lekcje

Jedno zestawienie wszystkich wyników na danych testowych. Najważniejsze jest porównanie z **baseline** — wynikiem „modelu", który zawsze wybiera najliczniejszą klasę.



Wykres: accuracy na zbiorze testowym (pełne kółko) vs baseline (puste kółko); po prawej różnica w punktach procentowych. K-Means — po dopasowaniu nazw klastrów (obliczone). Regresja liniowa nie ma accuracy, dlatego jest tylko w tabeli.

	NOTEBOOK	DANE	WYNIK NA TEŚCIE	BASELINE	BALANCED ACC.	WERDYKT
A1	**Regresja liniowa** regresja / LinearRegression	✓ czyste, syntetyczne	RMSE 8,93 $ · R^2 ≈ 0,99	RMSE ≈ 79 $	–	✓ bardzo dobry
A2	**Regresja logistyczna** klasyfikacja / LogisticRegression	✓ zbalansowane, separowalne	90,6%	50,9%	0,91	✓ bardzo dobry
A3	**KNN** klasyfikacja / KNN, K=24	! klasy się nakładają	85,0% (K=1: 72,7%)	51,3%	0,85	✓ dobry, lekko optymistyczny
A4	**Drzewo decyzyjne** klasyfikacja	! 16% klasy 1, słaby sygnał	73,0% · recall(1) 0,24	84,6%	0,53	✗ poniżej baseline
A4	**Las losowy** klasyfikacja	! jw.	84,55% · recall(1) 0,02	84,59%	0,51	✗ = baseline
A5	**SVM** klasyfikacja 3 klas / SVC	✓ wzorcowe	95,6% / 93,3%	37,8%	0,96 / 0,94	✓ bardzo dobry
A6	**K-Means** klastrowanie	! błędy, różne skale	77,9% po dopasowaniu (notebook: 22,1%)	72,7%	0,64	✗ słaby
A7	**NLP model 1** tekst / Count + NB	! niezbalansowane	92,6% · recall(1★) 0,70	81,4%	0,84	✓ dobry
A7	**NLP model 2** tekst / TF-IDF + NB	! jw.	81,4% · recall(1★) 0	81,4%	0,50	✗ model stały
A8	**Keras** sieć neuronowa	! braki, 80/20	88,1% · recall(Charged Off) 0,46	80,1%	0,72	! umiarkowany

Najważniejsze lekcje z 8 notebooków

1. **Zanim ucieszysz się z accuracy, policz baseline.** Las losowy (84,55%) i NLP z TF-IDF (81,4%) mają dokładnie tyle, ile „model" zawsze wybierający klasę większościową (A4, A7).

2. **Najpierw macierz pomyłek, potem raport.** Kolumna samych zer = klasa, której model nigdy nie przewiduje (A7). Raport można „upiększyć" parametrem `labels` — macierzy nie.
3. **Przy niezbalansowanych klasach liczy się mała klasa:** jej recall i precision oraz macro avg / balanced accuracy (A4, A7, A8).
4. **`describe()` → min i max każdej kolumny.** Wartości niemożliwe (Grad.Rate 118, PhD 103) widać od razu (A6).
5. **Metody oparte na odległości (KNN, K-Means, SVM, sieci) wymagają skalowania** — a skaler, vectorizer i imputację dopasowuje się tylko na danych treningowych (A3, A7 vs A8).
6. **Zbiór testowy to jednorazowy egzamin:** nie dobieraj na nim K, hiperparametrów ani liczby epok (A3, A8).
7. **Mały zbiór testowy = duża niepewność.** Różnica jednego przykładu na 45 to szum (A5).
8. **W klastrowaniu numery klastrów nic nie znaczą** — porównuj z etykietami przez najlepsze dopasowanie albo ARI (A6).
9. **EDA zapowiada wynik:** rozdzielone chmury na pairplocie → wysoka skuteczność (A2, A5); nakładające się rozkłady → słaby model (A4).
10. **Jedna trafiona predykcja niczego nie dowodzi** („nowy klient" w A8).

A10 Uwagi techniczne przy ponownym uruchamianiu

Część wyników w notebookach powstała na starszych wersjach bibliotek, a w metadanych kilku plików zapisane jest już środowisko Python 3.13. W nowych wersjach poniższe miejsca dadzą błąd albo inny wynik.

PROBLEM	GDZIE	NAPRAWA
`df.corr()` lub `groupby(…).mean()` na danych z kolumnami tekstowymi → ValueError / TypeError (pandas ≥ 2.0)	Keras In [11], [12], [27], [53], [54]; NLP In [11]	`df.corr(numeric_only=True)`, `groupby(…).mean(numeric_only=True)` albo wybór kolumny: `groupby('total_acc')['mort_acc'].mean()`
Zapis łańcuchowy `df['kol']['wiersz'] = …` nic nie zmienia (pandas ≥ 3.0, Copy-on-Write)	K-Means In [33]	`df.loc['Cazenovia College', 'Grad.Rate'] = 100`
Argumenty pozycyjne w seaborn (`lmplot('a', 'b', …)`, `kdeplot(x, y)`, `shade=`) → błąd w seaborn ≥ 0.12	K-Means In [28], [29]; SVM In [7]	`sns.lmplot(x='Room.Board', y='Grad.Rate', data=df, …)`; `sns.kdeplot(x=…, y=…, fill=True, thresh=0.05)`
`sns.distplot` przestarzałe	Keras In [10]	`sns.histplot(df['loan_amnt'], bins=40)`
`model.predict_classes` usunięte (TensorFlow ≥ 2.6)	Keras In [99], [103]	`(model.predict(X) > 0.5).astype(int)`
Inny układ `classification_report` („avg / total")	KNN In [30], [33]	to tylko format starszej wersji; dziś: accuracy / macro avg / weighted avg
Brak `random_state` → inne liczby po każdym uruchomieniu	KNN In [23]; SVM In [9]; K-Means In [37]; drzewa In [18], [25]	dodaj `random_state=…` w podziale i w modelu, jeśli chcesz porównywać wyniki
Komórki wykonane nie po kolei → NameError / KeyError	Keras In [25], [26], [29]	*Kernel → Restart & Run All*

TEIL B

auf Deutsch

Interpretation von 8 Notebooks: Daten und Training · Ergebnisse auf Testdaten · wo man hinschaut

B0 So liest du dieses Dokument + Spickzettel

Für jedes der 8 Notebooks beantworte ich zwei Fragen: **(1) Waren die Daten für das Training geeignet und wie lief das Training?** und **(2) Wie interpretiert man die Ergebnisse auf den Testdaten** — also auf Daten, die das Modell beim Lernen nicht gesehen hat? Jede Schlussfolgerung verweist auf die Stelle im Notebook, an der man sie sieht.

Aufbau jedes Kapitels

- **Steckbrief** — Datei, Aufgabe, Modell, Problemtyp, Daten.
- **Kurz gesagt** — drei Bewertungen: Daten / Training / Testergebnis (✓ in Ordnung, ! mit Einschränkungen, ✗ Problem).
- **1) Daten und Training — wo man hinschaut** — „Belegzeilen": links Zelle und Code, rechts *was zu sehen ist* → *was es bedeutet*.
- **2) Ergebnisse auf den Testdaten — wie man sie liest** — Konfusionsmatrix, Metriken, Vergleich mit der Baseline.
- **Vorsicht**, manchmal ein **Zusatztest**, am Ende das **Fazit**.

Verweise und Kennzeichnungen

`In [5]` ist der Ausführungszähler links neben der Zelle in Jupyter. Daneben steht ein Codeausschnitt, damit du die Zelle mit Strg+F findest. Führst du das Notebook neu aus, ändern sich die Nummern — dann nach dem Code suchen. Im Keras-Notebook wiederholen sich Nummern (die Datei enthält Ausgaben aus zwei Läufen), dort zählt der Codeausschnitt mehr.

berechnet — die Zahl steht nicht direkt im Notebook; ich habe sie aus den sichtbaren Ausgaben berechnet (z. B. die Baseline aus der Spalte `support`). Zusatztest — eigener Lauf des Codes mit denselben, öffentlich verfügbaren Daten (K-Means und NLP). Die Notebook-Ergebnisse wurden dabei exakt reproduziert — es sind also sicher dieselben Daten.

Spickzettel 1 — sind die Daten fürs Training geeignet?

FRAGE	WO HINSCHAUEN	WARNSIGNAL
Wie viele Daten, sind sie vollständig?	`df.info()`: Zeilenzahl, *Non-Null Count*, *Dtype*	*Non-Null* kleiner als die Zeilenzahl = fehlende Werte; `object` = Text, der kodiert werden muss
Sind die Werte möglich?	`df.describe()`: Zeilen *min* und *max*	Prozent > 100, negative Zeiten oder Beträge, *max* ein Vielfaches des 75-%-Quantils
Haben die Merkmale ähnliche Skalen?	`describe()`: *std*, *min*/*max*	Unterschiede ×100–×1000 → Skalierung bei KNN, K-Means, SVM, neuronalen Netzen
Sind die Klassen ausgewogen?	`countplot`, `value_counts()`, Mittelwert einer 0/1-Spalte	eine Klasse ≥ 70–80 % → Accuracy täuscht, Baseline berechnen
Trennen die Merkmale die Klassen?	`pairplot`, Histogramme und Boxplots mit `hue=Label`, `corr()` mit dem Ziel	Klassenverteilungen fast deckungsgleich → schwaches Modell

FRAGE	WO HINSCHAUEN	WARNSIGNAL
Gibt es ein Datenleck?	Merkmalsliste; Reihenfolge von `fit` und `train_test_split`	Merkmal erst „hinterher“ bekannt; Scaler / Vectorizer / Imputation vor dem Split angepasst
Ist die Bewertung fair?	`train_test_split` : `test_size` , `random_state` , `stratify`	kein `random_state` (nicht reproduzierbar); Testdaten fürs Tuning benutzt

Spickzettel 2 — wie lief das Training?

- In scikit-learn gibt `fit()` nichts aus. Über die Qualität sagen erst die Ergebnisse etwas. Schnelltest: `model.score(X_train, y_train)` mit `model.score(X_test, y_test)` vergleichen. **Großer Unterschied** = Überanpassung (Overfitting); **beide niedrig** = Unteranpassung (Modell zu einfach oder kein Signal in den Daten).
- Lernkurven (Keras): *loss* und *val_loss* fallen gemeinsam → gut; *val_loss* stagniert oder steigt, während *loss* weiter fällt → Overfitting; beide hoch → Underfitting.
- Plot „Fehler gegen Hyperparameter“ (KNN): Wert aus dem flachen Minimum wählen, nicht aus einem einzelnen „Loch“.

Spickzettel 3 — Ergebnisse auf Testdaten lesen

	vorhergesagt 0	vorhergesagt 1
wahr 0	**TN** richtig 0	**FP** Fehlalarm
wahr 1	**FN** übersehen	**TP** richtig 1

In scikit-learn gilt: **Zeilen = Wahrheit, Spalten = Vorhersage**. Die Klassen stehen in sortierter Reihenfolge der Labels (0, 1 oder 1, 5 oder alphabetisch). Diagonale = Treffer.

- **accuracy** = (TN + TP) / alle
- **precision** (Klasse 1) = TP / (TP + FP) — *Wenn das Modell „1“ sagt, wie oft stimmt es?*
- **recall** (Klasse 1) = TP / (TP + FN) — *Welchen Anteil der echten „1“ findet es?*
- **F1** = 2·P·R / (P + R) — Kompromiss aus Precision und Recall
- **support** = Anzahl echter Beispiele der Klasse im Testset
- **macro avg** = einfacher Mittelwert über die Klassen (jede Klasse gleich wichtig); **weighted avg** = mit support gewichtet (von der großen Klasse dominiert)
- **balanced accuracy** = Mittelwert der Recalls aller Klassen; bei 2 Klassen ist 0,5 = Raten

- **Baseline** = Anteil der größten Klasse im Testset (größter *support* / Summe). So viel Accuracy hat ein „Modell“, das immer dasselbe sagt. Ein Modell ist erst deutlich darüber etwas wert.
- **Unsicherheit** der Accuracy: etwa ±1,96·√(p·(1−p)/n). Für n = 45 sind das ±6 PP, n = 300: ±4, n = 1.226: ±1,5, n = 7.905: ±0,7. Kleinere Unterschiede sind Rauschen.
- **Regression**: MAE (mittlerer absoluter Fehler in Einheiten des Ziels), MSE (Quadrate — bestraft große Fehler stärker), RMSE (Wurzel aus MSE, wieder in Zieleinheiten), R^2 (Anteil erklärter Streuung: 1 = perfekt, 0 = nicht besser als der Mittelwert). RMSE mit der Standardabweichung des Ziels vergleichen. Residuen (Wahrheit − Vorhersage) sollten symmetrisch um 0 und ohne Muster sein.
- **Clustering**: Cluster-Nummern sind zufällig. Mit Labels vergleicht man über die beste Zuordnung oder mit einem nummerierungsunabhängigen Maß (Adjusted Rand Index).

REIHENFOLGE BEIM LESEN EINES KLASSIFIKATIONSERGEBNISSES

1. Konfusionsmatrix: Wird jede Klasse überhaupt vorhergesagt? (Spalte nur mit Nullen = nein)
2. Accuracy vs. Baseline.
3. Recall und Precision der wichtigsten Klasse — meist der kleineren.
4. Macro avg / balanced accuracy.
5. Größe des Testsets (Unsicherheit).
6. Erst dann geschäftliche Schlüsse.

B1 Lineare Regression — E-Commerce-Kunden

Datei	3-Lineare_Regression-Projekt_Uebung-Loesung.ipynb
Aufgabe	die jährlichen Ausgaben eines Kunden (`Yearly Amount Spent` , in $) aus 4 Verhaltensmerkmalen vorhersagen
Modell	`LinearRegression` (scikit-learn)
Typ	Regression (Ergebnis ist eine Zahl, keine Klasse)
Daten	500 Kunden; 4 numerische Merkmale + 3 Textspalten (nicht genutzt)

KURZ GESAGT

✓ **Daten**	Sehr gut: vollständig, sauber, mit klaren linearen Zusammenhängen. Die Daten sind künstlich erzeugt — daher so „perfekt“.
✓ **Training**	Korrekter 70/30-Split mit `random_state=101` . Einfaches Modell (5 Parameter) auf 350 Beispielen — Overfitting praktisch ausgeschlossen.
✓ **Testergebnis**	Sehr gut: MAE ≈ 7,23 $, RMSE ≈ 8,93 $ bei durchschnittlich 499 $ Ausgaben; R^2 ≈ 0,99 (berechnet). Die Residuen sehen normalverteilt aus.

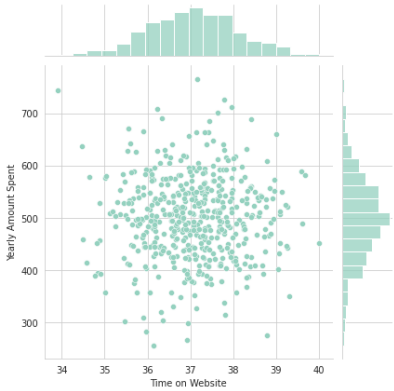
1) Daten und Training — wo man hinschaut

`In [5]` `customers.info()`	**Zu sehen:** 500 Zeilen, in jeder Spalte „500 non-null“; 5 Spalten `float64` und 3 `object` (Email, Address, Avatar). → **Folgerung:** Keine fehlenden Werte. Die Textspalten sind Kennungen bzw. zufällige Attribute — sie kommen zu Recht nicht ins Modell (In [14] wählt nur die 4 numerischen Merkmale).
`In [4]` `customers.describe()`	**Zu sehen:** Die vier Merkmale haben *std* ≈ 1,0 (0,99 / 0,99 / 1,01 / 1,00), Mittelwert ≈ Median (z. B. 33,05 vs. 33,08). Plausible Wertebereiche (Length of Membership 0,27–6,92 Jahre, keine negativen Werte). → **Folgerung:** Symmetrische Verteilungen, keine absurden Werte. So gleiche *std*-Werte deuten auf generierte Daten hin. Wichtige Folge: Alle Merkmale haben dieselbe Skala, daher darf man später die Koeffizienten direkt vergleichen.
`In [7]` `jointplot(x='Time on Website', …)`	**Zu sehen:** Runde Punktwolke ohne Steigung. → **Folgerung:** Die Zeit auf der Website hängt nicht mit den Ausgaben zusammen. Der Code-Kommentar „*# Je mehr Zeit auf der Webseite, desto mehr Ausgaben*“ widerspricht dem Plot (und dem späteren Koeffizienten 0,19). Regel: Plots und Zahlen glauben, nicht Kommentaren.
`In [8]` `jointplot(x='Time on App', …)`	**Zu sehen:** Deutlich ansteigende Wolke. → **Folgerung:** Mehr Zeit in der App geht mit höheren Ausgaben einher.
`In [10] / In [11]` `sns.pairplot(customers)`	**Zu sehen:** In der Zeile „Yearly Amount Spent“ liegt die schmalste, schräge Wolke bei Length of Membership; die Eingangsmerkmale untereinander bilden runde Wolken. → **Folgerung:** Stärkster Prädiktor ist die Mitgliedschaftsdauer (so antwortet auch das Notebook in In [11]). Merkmale untereinander unkorreliert → keine Multikollinearität, stabile Koeffizienten.
`In [12]` `lmplot(x='Length of Membership', …)`	**Zu sehen:** Punkte nah an der Linie, sehr schmales Konfidenzband. → **Folgerung:** Linearer Zusammenhang — die lineare Regression passt.
`In [16]` `train_test_split(X, y, test_size=0.3, random_state=101)`	**Zu sehen:** 350 Trainingszeilen (bestätigt durch In [22] `print(X_train.head)` : „[350 rows x 4 columns]“) und 150 Testzeilen. → **Folgerung:** Sinnvoller Split; `random_state` macht das Ergebnis reproduzierbar.

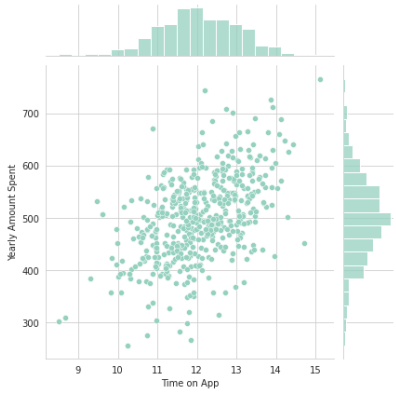
In [19]

```
lm.fit(X_train, y_train)
```

Zu sehen: Nur `LinearRegression()` — sklearn gibt keine Trainingsmetriken aus. → **Folgerung:** Den Trainingsverlauf beurteilt man indirekt: 5 Parameter auf 350 Beispielen = kein Overfitting-Risiko. Prüfen: `lm.score(X_train, y_train)` und `lm.score(X_test, y_test)` sollten fast dasselbe R^2 liefern.



In [7] Time on Website vs. Yearly Amount Spent — Wolke ohne Trend: kein Zusammenhang.



In [8] Time on App vs. Yearly Amount Spent — klarer steigender Trend.

2) Ergebnisse auf den Testdaten — wie man sie liest

In [22]

```
plt.scatter(y_test, predictions)
```

Zu sehen: Die Punkte liegen fast perfekt auf der Diagonale. → **So liest du es:** Jeder Punkt ist ein Testkunde: x-Achse echte, y-Achse vorhergesagte Ausgaben. Perfektes Modell = Linie y = x. Schmales Band = kleine Fehler. Eine Krümmung hieße Nichtlinearität, ein sich nach rechts öffnender „Trichter" wachsende Fehler bei großen Werten. Nichts davon ist hier zu sehen.

In [23]

```
metrics.mean_absolute_error(…) / mean_squared_error(…)
```

Zu sehen: MAE 7,23; MSE 79,81; RMSE 8,93. → **So liest du es:** MAE — durchschnittlicher Fehler 7,23 $; RMSE — „typischer" Fehler 8,93 $ (bestraft große Fehler stärker); MSE — dasselbe in $², schwer interpretierbar. Bezugspunkt: Durchschnitt 499 $, std 79 $ (In [4]). Ein Modell, das immer den Mittelwert ausgibt, hätte RMSE ≈ 79 $ — unseres liegt etwa 9-mal seltener daneben, und der MAE entspricht rund 1,5 % der mittleren Ausgaben. RMSE nur wenig größer als MAE → keine einzelnen riesigen Fehler.

MAE	RMSE	R^2 (Schätzung)	Testset
7,23 $	**8,93 $**	**≈ 0,99**	**150**
≈ 1,5 % des Mittelwerts	vs. ≈ 79 $ für „Mittelwert"	berechnet	Kunden

Abschnitt „Das Modell bewerten"

Zu sehen: Der Text kündigt R^2 an („erklärte Varianz"), der Code berechnet es aber nicht. → **Folgerung:** Schätzung (berechnet): $R^2 \approx 1 - \text{MSE}/\text{Var}(y) = 1 - 79{,}8 / 79{,}3^2 \approx 0{,}987$ — das Modell erklärt also rund 99 % der Streuung der Ausgaben. Exakt: `metrics.r2_score(y_test, predictions)`.

In [30]

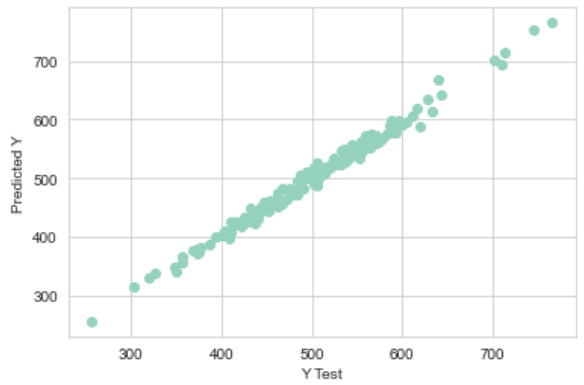
```
sns.histplot((y_test-predictions), bins=50)
```

Zu sehen: Glockenform, Mitte bei 0, Bereich ca. −27…+30 $. → **So liest du es:** Residuum = Wahrheit − Vorhersage. Mitte bei 0 → kein systematischer Fehler in eine Richtung; Symmetrie und keine langen „Schwänze" → Annahmen der linearen Regression erfüllt. Sinnvolle Ergänzung: Plot Residuum vs. Vorhersage (sollte eine formlose Wolke sein).

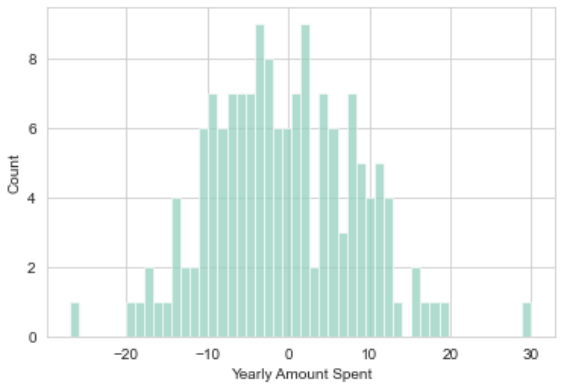
In [20] / In [25]

```
lm.coef_ / pd.DataFrame(lm.coef_, X.columns)
```

Zu sehen: Length of Membership 61,28; Time on App 38,59; Avg. Session Length 25,98; Time on Website 0,19. → **So liest du es:** Bei sonst gleichen Merkmalen: +1 Jahr Mitgliedschaft ≈ +61 $ pro Jahr, +1 min in der App ≈ +39 $, +1 min Beratung im Laden ≈ +26 $, +1 min auf der Website ≈ +0,19 $ (praktisch null). Der direkte Vergleich dieser Zahlen ist nur zulässig, weil alle Merkmale *std* ≈ 1 haben (In [4]).



In [22] echte (x) vs. vorhergesagte (y) Ausgaben — Punkte auf der Diagonale.



In [30] Histogramm der Residuen — symmetrisch, um 0 zentriert.

VORSICHT

- So ein gutes Ergebnis wäre bei echten Daten verdächtig (z. B. weil die Zielvariable in die Merkmale „sickert"). Hier liegt es daran, dass die Daten aus einer linearen Formel mit wenig Rauschen erzeugt wurden.
- Korrelation ≠ Kausalität: Das Modell sagt nicht, dass „erzwungene" längere App-Nutzung die Ausgaben erhöht.
- scikit-learn liefert keine Signifikanz der Koeffizienten (p-Werte). Ob sich 0,19 für die Website von null unterscheidet, beantwortet `statsmodels` (OLS → `summary()`).

FAZIT

Gute Daten → problemloses Training → sehr gutes, verlässliches Testergebnis. Fürs Business (Abschnitt „Schlussfolgerungen"): Die Ausgaben erklären vor allem Mitgliedschaftsdauer und App-Zeit; die Website hat keinen Einfluss. Ob man die App ausbaut oder die Website verbessert, entscheiden die Daten allein nicht (Kosten, Kausalität) — das Notebook weist zu Recht darauf hin.

B2 Logistische Regression — Klicks auf Werbung

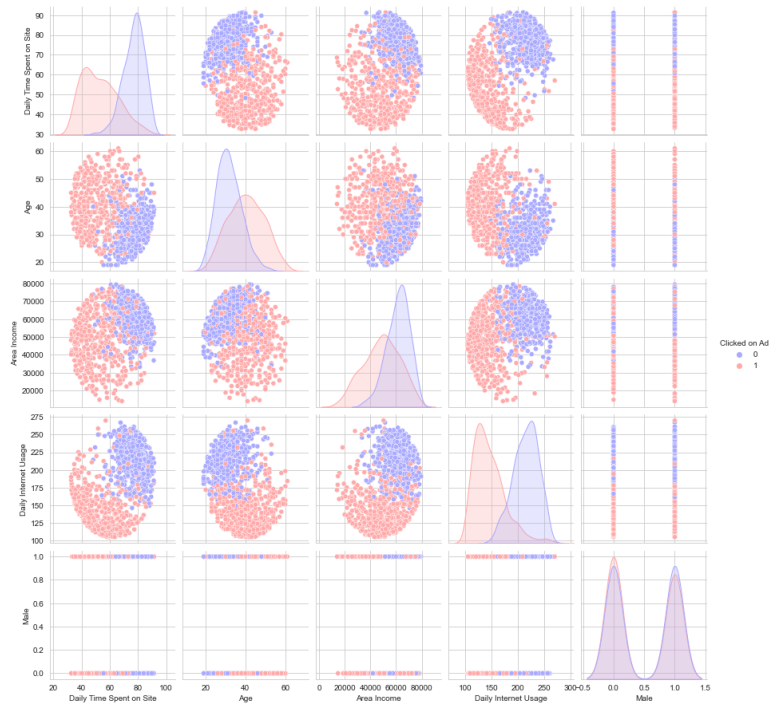
Datei	3-Logistische_Regression_Projekt-Loesung.ipynb
Aufgabe	vorhersagen, ob ein Nutzer auf eine Anzeige klickt (`Clicked on Ad` : 1 = ja, 0 = nein)
Modell	`LogisticRegression`
Typ	binäre Klassifikation
Daten	1.000 Nutzer, 10 Spalten (Fake-Daten)

KURZ GESAGT

✓ **Daten**	Sehr gut: vollständig, Klassen perfekt ausgewogen (500/500), die Merkmale trennen die Klassen deutlich.
✓ **Training**	Korrekter 67/33-Split (`random_state=42`). Kleiner Hinweis: keine Skalierung trotz sehr unterschiedlicher Wertebereiche.
✓ **Testergebnis**	Accuracy 0,91 bei Baseline 0,51. Asymmetrische Fehler: 25 übersehene Klicker vs. 6 Fehlalarme berechnet.

1) Daten und Training — wo man hinschaut

In [4] `ad_data.info()`	**Zu sehen:** 1.000 Zeilen, überall „1000 non-null"; 4 `object`-Spalten (Ad Topic Line, City, Country, Timestamp). → **Folgerung:** Keine fehlenden Werte. Die Textspalten haben Hunderte eindeutiger Werte — in In [17] bewusst weggelassen (für ein einfaches Modell richtig; aus Timestamp könnte man z. B. die Tageszeit ableiten).
In [5] `ad_data.describe()`	**Zu sehen:** Clicked on Ad: *mean* 0,50 → genau 500 × 0 und 500 × 1. Verdächtig „runde" Mittelwerte (65,0002; 55.000,00008; 180,0001). Age 19–61, Area Income ca. 14.000–79.000. → **Folgerung:** Ausgewogene Klassen → Accuracy ist hier ein faires Maß. Die Daten sind generiert. Sehr unterschiedliche Skalen (Einkommen ~55.000, Alter ~36) — siehe „Vorsicht".
In [7] `ad_data['Age'].hist(bins=30)`	**Zu sehen:** Alter von 19 bis 61 Jahren, meist 25–45. → **Folgerung:** Plausible Verteilung, keine unmöglichen Werte.
In [15] `sns.pairplot(ad_data, hue='Clicked on Ad')`	**Zu sehen:** (wichtigster Plot) Daily Time Spent on Site und Daily Internet Usage bilden zwei klar getrennte Wolken; auf der Diagonale zwei verschobene „Buckel". Klicker sind älter (Age) und kommen aus Regionen mit niedrigerem Einkommen (Area Income). Male: identische Verteilungen. → **Folgerung:** Schon vor dem Training ist klar: Die Aufgabe ist leicht — man kann etwa 90 % Accuracy erwarten. Male trägt keine Information.
In [13] `jointplot(x='Daily Time Spent on Site', y='Daily Internet Usage')`	**Zu sehen:** Zwei Punktwolken. → **Folgerung:** Das sind dieselben zwei Gruppen (Klicker / Nicht-Klicker) — gute Trennbarkeit.
In [18] `train_test_split(X, y, test_size=0.33, random_state=42)`	**Zu sehen:** Testset mit 330 Beispielen (support 162 + 168 in In [23]). → **Folgerung:** Klassenverhältnis im Test weiterhin ca. 50/50; reproduzierbar.
In [20] `LogisticRegression().fit(X_train, y_train)`	**Zu sehen:** Standardeinstellungen, keine Skalierung; keine Warnungen in der Ausgabe. → **Folgerung:** Training abgeschlossen. Lineares Modell mit 6 Parametern auf 670 Beispielen — geringes Overfitting-Risiko.



In [15] Pairplot mit `hue='Clicked on Ad'` — Daily Time Spent on Site und Daily Internet Usage trennen die Klassen; Male (letzte Spalte) trennt nicht.

2) Ergebnisse auf den Testdaten — wie man sie liest

In [23]
```
print(classification_report(y_test, predictions))
```

Zu sehen: Klasse 0: precision 0,86, recall 0,96; Klasse 1: precision 0,96, recall 0,85; accuracy 0,91; macro ≈ weighted ≈ 0,91; support 162 / 168. → **So liest du es:** Klasse 1 (geklickt): Wenn das Modell „klickt" sagt, stimmt es in 96 % der Fälle (precision), es findet aber nur 85 % der tatsächlichen Klicker (recall) — jeden siebten übersieht es. Klasse 0: Fast alle Nicht-Klicker werden erkannt (96 %). Macro ≈ weighted, weil die Klassen gleich groß sind.

Baseline

Zu sehen: Größte Klasse im Test: 168/330 = 50,9 % berechnet. → **Folgerung:** Das Modell erreicht 90,6 % — rund 40 Prozentpunkte über „Raten".

Konfusionsmatrix aus dem Report rekonstruiert berechnet

	vorhergesagt 0	vorhergesagt 1
wahr: 0 (kein Klick)	**156**	6
wahr: 1 (Klick)	25	**143**

Das Notebook druckt sie nicht, sie folgt aber eindeutig aus precision, recall und support. Prüfen: `confusion_matrix(y_test, predictions)`.

Accuracy
90,6 %
299 von 330

Baseline
50,9 %
berechnet

Recall „Klick"
0,85
25 übersehen

Precision „Klick"
0,96
6 Fehlalarme

Matrix

Zu sehen: 6 Fehlalarme (Klick vorhergesagt, aber keiner) vs. 25 übersehene Klicks (geklickt, aber das Modell sagt „nein"). → **Folgerung:** Asymmetrische Fehler — das Modell ist beim Vorhersagen eines Klicks „vorsichtig".

Geschäftliche Schlüsse

Zu sehen: Precision 0,96 und recall 0,85 für Klasse 1. → **Folgerung:** Soll die Anzeige nur an Leute gehen, die klicken (sparsames Budget), ist die hohe Precision ein Vorteil. Darf man keine potenziellen Kunden verpassen, muss der Recall besser werden: Schwelle senken (`predict_proba(X_test)[:,1] > 0.4` statt 0,5). Bei 330 Testbeispielen liegt die Unsicherheit der Accuracy bei ca. ±3 PP (0,87–0,94) berechnet.

VORSICHT

- Keine Skalierung: Area Income (~55.000) und Age (~36) im selben Modell. Die L2-Regularisierung (Standard `C=1`) wirkt dann ungleich auf die Merkmale, und der Solver konvergiert evtl. langsamer. Besser: `make_pipeline(StandardScaler(), LogisticRegression())` .
- Das Notebook zeigt weder die Konfusionsmatrix noch die Koeffizienten (`logmodel.coef_`) — gerade sie zeigen, wo das Modell irrt und welche Merkmale in Richtung Klick „schieben".

FAZIT

Gute, gut trennbare Daten → korrektes Training → sehr gutes und verlässliches Testergebnis (die Klassen sind ausgewogen, also täuscht die Accuracy nicht). Einzige Schwäche: Jeder siebte Klicker wird übersehen.

B3 KNN — anonymisierte Daten

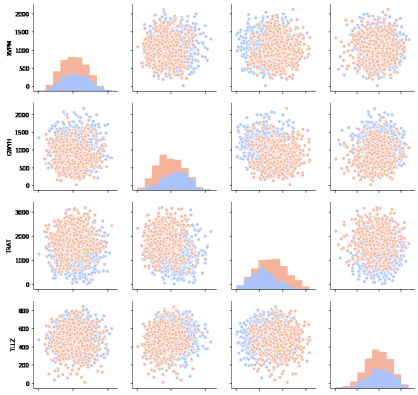
Datei	3-K_Nearest_Neighbors_Projekt-Loesung.ipynb
Aufgabe	`TARGET CLASS` (0/1) aus 10 anonymisierten Merkmalen vorhersagen
Modell	`KNeighborsClassifier`; Wahl von K mit der „Ellbogen"-Methode
Typ	binäre Klassifikation
Daten	ca. 1.000 Zeilen (abgeleitet aus 300 = 30 % Test), künstliche Daten

KURZ GESAGT

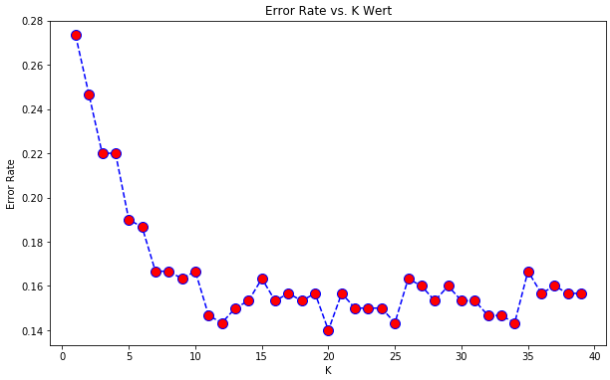
! **Daten**	Vermutlich in Ordnung, aber nicht geprüft (kein `info()` / `describe()`). Klassen ausgewogen, überlappen aber in jedem Merkmalspaar stark.
! **Training**	Skalierung vorhanden (bei KNN Pflicht), aber: Scaler auf allen Daten angepasst, Split ohne `random_state`, K auf den Testdaten gewählt.
✓ **Testergebnis**	K=1: 72,7 %, K=24: 85,0 % bei Baseline 51,3 %. Gutes Ergebnis, aber leicht optimistisch.

1) Daten und Training — wo man hinschaut

`In [3]` `df.head()`	**Zu sehen:** Spalten XVPM, GWYH, TRAT … (künstliche Namen) und sehr unterschiedliche Wertebereiche (z. B. TLLZ ca. 280–725, TRAT ca. 610–2640 in den ersten Zeilen). → **Folgerung:** Unterschiedliche Skalen, und KNN rechnet mit Abständen — Skalierung ist Pflicht.
fehlt `df.info() / df.describe()`	**Zu sehen:** Das Notebook springt direkt zum Plot. → **Folgerung:** Aus dem Notebook wissen wir nicht, ob es fehlende Werte oder Ausreißer gibt. Eine Lücke in der EDA — immer prüfen.
`In [4]` `sns.pairplot(df, hue='TARGET CLASS')`	**Zu sehen:** In jedem 2D-Plot überlappen die Wolken beider Klassen stark; die Verteilungen auf der Diagonale sind nur leicht verschoben (am stärksten z. B. TRAT, GWYH). → **Folgerung:** Kein einzelnes Merkmal und kein Paar trennt die Klassen — die Information ist über viele Dimensionen „verteilt". Man sollte ein mittleres Ergebnis und einen großen Einfluss von K erwarten. Die Balkenhöhen auf der Diagonale sagen nichts über die Klassengrößen (Histogramme gemeinsam gezeichnet); die Ausgewogenheit bestätigt erst support 154/146 in In [30].
`In [7] → In [9]` `scaler.fit(df.drop('TARGET CLASS', axis=1))`	**Zu sehen:** Nach dem Skalieren (In [9] `df_feat.head()`) liegen die Werte um 0 (ca. −1,8… 2,2). → **Folgerung:** Skalen angeglichen — gut. Aber ein methodischer Fehler: Der Scaler wurde auf ALLEN Daten vor dem Split angepasst (Mittelwert und std haben die Testdaten „gesehen"). Bei 1.000 Zeilen ist der Effekt minimal; korrekt: erst splitten, dann `scaler.fit(X_train)` und `transform` auf beiden Teilen (so macht es das Keras-Notebook).
`In [23]` `train_test_split(scaled_features, …, test_size=0.30)`	**Zu sehen:** Kein `random_state`. → **Folgerung:** Jeder Lauf = anderer Split und andere Zahlen. Nach erneutem Ausführen siehst du andere Ergebnisse als im Notebook — das ist normal, kein Fehler.
`In [25] / In [26]` `KNeighborsClassifier(n_neighbors=1).fit(…)`	**Zu sehen:** KNN lernt keine Parameter, es merkt sich die Trainingspunkte. → **Folgerung:** Bei K=1 wäre die Trainings-Accuracy 100 % (jeder Punkt ist sein eigener nächster Nachbar). Lehrbuch-Overfitting — die Wahrheit zeigt erst der Test.



In [4] Ausschnitt aus dem Pairplot (Merkmale XVPM, GWYH, TRAT, TLLZ) — die Klassenwolken überlappen.



In [32] Testfehler gegen K — steiler Abfall bis K≈11, danach flaches Plateau.

2) Ergebnisse auf den Testdaten — wie man sie liest

In [29] — K = 1

	vorh. 0	vorh. 1
wahr: 0	**113**	41
wahr: 1	41	**105**

In [33] — K = 24

	vorh. 0	vorh. 1
wahr: 0	**131**	23
wahr: 1	22	**124**

`In [29] / In [30]` `K = 1`	**Zu sehen:** 41 + 41 Fehler bei 300 → Accuracy 72,7 % (berechnet); Precision und Recall beider Klassen ca. 0,72–0,73 (symmetrische Fehler). → **Folgerung:** Im Training wären es 100 %, im Test 73 % → Overfitting: Das Modell reagiert auf das Rauschen einzelner Nachbarn.
`In [31] / In [32]` `Plot „Error Rate vs. K Wert“`	**Zu sehen:** Der Fehler fällt von ca. 0,27 (K=1) auf ca. 0,15 für K ≥ 11, dann „Plateau“ 0,14–0,165 mit zackigen Sprüngen. → **So liest du es:** K aus dem flachen Bereich wählen, nicht aus einem einzelnen „Loch“. 0,14 vs. 0,16 sind etwa 6 von 300 Beispielen — Rauschen. K=20 (Minimum ca. 0,14) ist nicht wirklich besser als K=24 (0,15).
`In [31]` `pred_i = knn.predict(X_test)`	**Zu sehen:** Die Schleife misst den Fehler auf `X_test` / `y_test` — der Test dient also zur Wahl von K. → **Folgerung:** Das Testset ist nicht mehr „unberührt“ — die finalen 85 % sind leicht geschönt. Korrekt: `GridSearchCV(KNeighborsClassifier(), {'n_neighbors': range(1, 40)}, cv=5)` auf den Trainingsdaten, Test nur einmal ganz am Ende.
`In [33]` `K = 24`	**Zu sehen:** 255/300 = 85,0 % (berechnet); Precision und Recall beider Klassen ca. 0,85; fast symmetrische Fehler (23 vs. 22). → **Folgerung:** Das Modell bevorzugt keine Klasse. Baseline 154/300 = 51,3 % (berechnet) → Modell +34 PP; Unsicherheit ca. ±4 PP.
`In [30] / In [33]` `Zeile „avg / total“`	**Zu sehen:** Anderes Report-Layout als in den übrigen Notebooks. → **Folgerung:** Format einer älteren scikit-learn-Version; heute siehst du *accuracy / macro avg / weighted avg*.

K = 1	K = 24	Baseline	Unsicherheit
72,7 %	**85,0 %**	**51,3 %**	**±4 PP**
Overfitting	255 von 300	berechnet	berechnet, n = 300

VORSICHT

- Drei kleine methodische Fehler auf einmal: Scaler vor dem Split, kein `random_state`, Wahl von K auf dem Test. Jeder für sich ist klein; zusammen machen sie das Ergebnis etwas optimistisch und nicht reproduzierbar.
- Beim erneuten Ausführen andere Zahlen erwarten (für K im Plateau etwa 82–88 %) — Folge des zufälligen Splits.

FAZIT

Schwierigere Daten (die Klassen überlappen), aber nach Skalierung und Wahl von K funktioniert das Modell ordentlich: 85 % bei 51 % Baseline. K=1 ist ein Lehrbuchbeispiel für Overfitting, der Plot „Fehler gegen K“ die Lehrbuchmethode, es zu erkennen.

B4 Entscheidungsbaum und Random Forest — LendingClub

Datei	3-Decision_Trees_und_Random_Forests_Projekt-Loesung.ipynb
Aufgabe	vorhersagen, ob ein Kredit NICHT vollständig zurückgezahlt wird (`not.fully.paid` = 1)
Modelle	`DecisionTreeClassifier()` (ohne Begrenzung) und `RandomForestClassifier(n_estimators=600)`
Typ	binäre Klassifikation, unausgewogene Klassen
Daten	9.578 LendingClub-Kredite 2007–2010; 13 Merkmale + Ziel

KURZ GESAGT

! **Daten**	Technisch sauber (keine fehlenden Werte), aber schwierig: nur 16 % nicht zurückgezahlte Kredite, und die Merkmale unterscheiden sie kaum (EDA).
! **Training**	Baum ohne Begrenzung → Overfitting. Der Random Forest hat praktisch gelernt, „zahlt zurück“ zu sagen.
✗ **Testergebnis**	Beide Modelle taugen nicht zur Risikoerkennung: Baum 73 % (unter der Baseline von 84,6 %), Forest 84,55 % = Baseline, er erkennt 2 % der faulen Kredite.

1) Daten und Training — wo man hinschaut

`In [3]` `loans.info()`	**Zu sehen:** 9.578 Zeilen, 14 Spalten, überall „9578 non-null“; eine `object` -Spalte: purpose. → **Folgerung:** Keine fehlenden Werte (die Daten wurden vorher bereinigt — sagt die Einleitung). purpose muss in Zahlen umgewandelt werden: In [13] `pd.get_dummies(…, drop_first=True)` , In [14] zeigt 19 Spalten.
`In [4]` `loans.describe() – Spalte not.fully.paid`	**Zu sehen:** *mean* = 0,160. → **Folgerung:** Die wichtigste Zahl im Notebook: Nur 16 % der Kredite sind Klasse 1. Ein Modell „immer 0“ hat schon ca. 84 % Accuracy (= Baseline). Accuracy wird hier täuschen — man muss auf Recall und Precision der Klasse 1 schauen.
`In [4]` `loans.describe() – min/max`	**Zu sehen:** Revol.bal max 1.207.359 bei einem Mittelwert von 16.914; inq.last.6mths max 33; delinq.2yrs max 13; revol.util max 119. → **Folgerung:** Stark schiefe Verteilungen mit Ausreißern (revol.util > 100 % ist bei überzogenem Limit möglich, sollte aber geprüft werden). Für Bäume ein kleines Problem — sie teilen an Schwellen, Skala und Ausreißer ändern wenig; Skalierung unnötig.
`In [6]` `hist FICO nach credit.policy`	**Zu sehen:** Alle mit credit.policy = 1 haben einen FICO ab ca. 660; darunter gibt es nur credit.policy = 0. → **Folgerung:** Credit.policy ist weitgehend eine Schwellenregel auf FICO — zwei Merkmale tragen teilweise dieselbe Information.
`In [7]` `hist FICO nach not.fully.paid`	**Zu sehen:** Die Verteilungen beider Gruppen decken sich fast; nicht zurückgezahlte sind nur leicht zu niedrigerem FICO verschoben. → **Folgerung:** FICO allein unterscheidet die Klassen schwach.
`In [8]` `countplot(x='purpose', hue='not.fully.paid')`	**Zu sehen:** Die blauen Balken (nicht zurückgezahlt) sind in jeder Kategorie klein; small_business hat einen deutlich höheren Anteil (ca. ¼), credit_card und major_purchase den niedrigsten (ca. 1/10). → **Folgerung:** Der Kreditzweck trägt etwas Information, aber wenig.

`In [9] / In [10]`

```
lmplot(y='int.rate',
x='fico',
col='not.fully.paid')
```

Zu sehen: Starker fallender Trend (höherer FICO → niedrigerer Zins), aber die Panels für not.fully.paid = 0 und = 1 sehen ähnlich aus. → **Folgerung:** Keine klare Trennung. Fazit der EDA: schwaches Signal, kleine Zielklasse → mit schwachen Ergebnissen rechnen.

`In [16]`

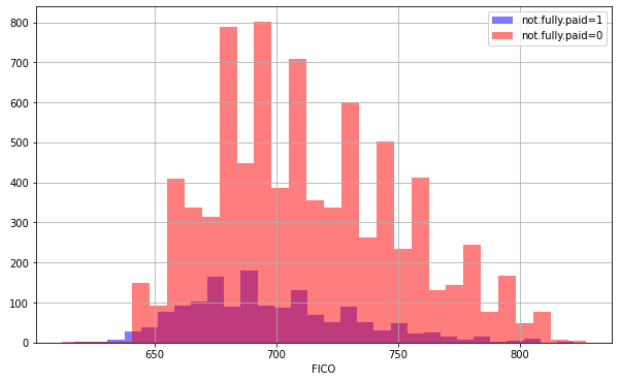
```
train_test_split(X, y,
test_size=0.30,
random_state=101)
```

Zu sehen: Test: 2.874 Kredite (support 2.431 + 443). → **Folgerung:** Im Test 15,4 % Klasse 1 — wie in den Gesamtdaten (ohne `stratify`, aber die Anteile passen trotzdem).

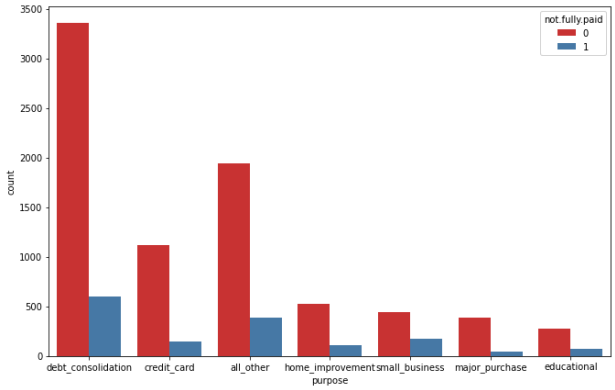
`In [18] / In [25]`

```
DecisionTreeClassifier()
/
RandomForestClassifier(n_
estimators=600)
```

Zu sehen: Standardeinstellungen: Baum ohne Tiefenbegrenzung, kein `class_weight`, kein `random_state`. → **Folgerung:** Der Baum wächst, bis fast jedes Blatt „rein" ist → er lernt die Trainingsdaten auswendig (Trainings-Accuracy ≈ 100 %; prüfen mit `dtree.score(X_train, y_train)`). Ohne `random_state` unterscheiden sich die Ergebnisse bei jedem Lauf etwas.



In [7] FICO nach not.fully.paid — Verteilungen fast deckungsgleich.



In [8] Kreditzweck nach not.fully.paid — Klasse 1 (blau) überall klein.

2) Ergebnisse auf den Testdaten — wie man sie liest

In [23] — Entscheidungsbaum

	vorh. 0	vorh. 1
wahr: 0 (bezahlt)	**1993**	438
wahr: 1 (nicht bezahlt)	338	**105**

In [30] — Random Forest

	vorh. 0	vorh. 1
wahr: 0 (bezahlt)	**2421**	10
wahr: 1 (nicht bezahlt)	434	**9**

`In [22] / In [23]`

```
Entscheidungsbaum
```

Zu sehen: Von 443 nicht zurückgezahlten Krediten fand der Baum 105 (recall 0,24); von 543 Alarmen „zahlt nicht" waren 105 richtig (precision 0,19). → **So liest du es:** Zufälliges Markieren ergäbe eine Precision von ca. 0,15 (Anteil der Klasse 1) — der Baum ist nur etwa 1,25-mal besser als Zufall. Accuracy 0,73 liegt UNTER der Baseline von 0,846. Macro avg F1 0,53 ≈ Zufallsniveau.

`In [29] / In [30]`

```
Random Forest
```

Zu sehen: Accuracy 0,85 sieht gut aus, aber (2.421 + 9) / 2.874 = 84,55 %, während „immer 0" 2.431 / 2.874 = 84,59 % ergibt berechnet. → **So liest du es:** Der Forest ist um EIN Beispiel schlechter als ein Modell, das immer „zahlt zurück" sagt. Klasse 1 hat er nur 19-mal vorhergesagt (9 davon richtig): recall 0,02 → er erkennt 2 % der faulen Kredite. Balanced accuracy 0,51 berechnet (0,50 = Raten).

Baseline	Baum	Random Forest	Balanced acc.
84,6 %	**73,0 %**	**84,55 %**	**0,53 / 0,51**
berechnet: „immer 0"	recall(1) 0,24	recall(1) 0,02	berechnet

warum „0,85“ täuscht	**Zu sehen:** Bei 85 % einer Klasse misst die Accuracy vor allem das Erkennen der Mehrheitsklasse. → **Folgerung:** Immer auf die Zeile der Klasse 1 im Report und auf macro avg schauen (F1 des Forests: 0,48).
`In [31]` `# Kommt darauf an welche Metrik wir optimieren`	**Zu sehen:** Das Notebook antwortet, es hänge von der Metrik ab und keines der Modelle sei gut gewesen. → **Folgerung:** Nach Accuracy gewinnt der Forest (aber = Baseline); nach Recall der Klasse 1, also der Risikoerkennung, die für Investoren zählt, gewinnt der Baum (0,24 vs. 0,02). Keines ist einsetzbar.
Frage vor In [28] „Erhälst du etwas komisches oder eine Art Warnung?“	**Zu sehen:** Report des Forests ohne Warnung. → **Folgerung:** „komisch“ ist genau, dass der Forest Klasse 1 fast nie vorhersagt. Hätte er sie nie vorhergesagt, gäbe sklearn eine *UndefinedMetricWarning* aus (precision = 0/0).

VORSICHT / WIE MAN ES VERBESSERT

- `class_weight='balanced'` (Baum und Forest) — Fehler bei Klasse 1 kosten mehr.
- Baum begrenzen: `max_depth`, `min_samples_leaf` — weniger Overfitting.
- Statt `predict()`: `predict_proba()` und eigene Schwelle (z. B. 0,2–0,3), gewählt an der Precision-Recall-Kurve; Bewertung mit ROC-AUC / Average Precision.
- Kreuzvalidierung (`StratifiedKFold`) statt eines einzigen Splits.

FAZIT

Saubere, aber wenig informative und unausgewogene Daten → Baum überangepasst, Forest zu „zahlt immer zurück“ degeneriert. Die hohe Accuracy des Forests ist eine Illusion — ein Lehrbuchbeispiel dafür, warum man immer mit der Baseline vergleichen muss.

B5 SVM — Iris

Datei	3-Support_Vector_Machines_Projekt-Loesung.ipynb
Aufgabe	die Irisart (setosa / versicolor / virginica) aus 4 Blütenmaßen bestimmen
Modell	`SVC()` (RBF-Kernel) + `GridSearchCV` für C und gamma
Typ	Mehrklassen-Klassifikation (3 Klassen)
Daten	Iris-Datensatz: 150 Blüten, je 50 pro Art

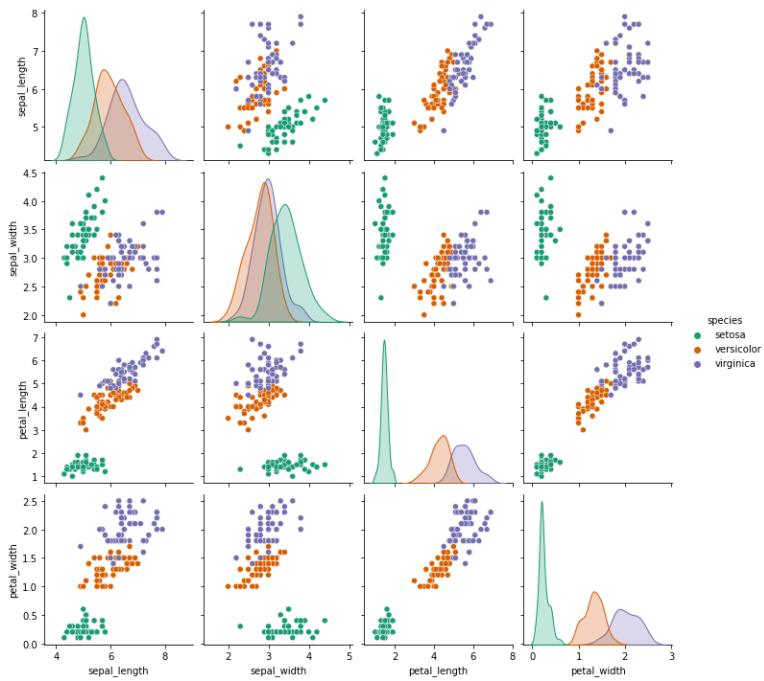
KURZ GESAGT

✓ **Daten**	Vorbildlich: vollständig, ausgewogen; setosa perfekt getrennt, versicolor und virginica überlappen leicht.
✓ **Training**	Korrekt; GridSearch mit 5-facher Kreuzvalidierung. Es fehlt die Ausgabe von `best_params_`.
✓ **Testergebnis**	43/45 (95,6 %) Standard-SVC, 42/45 (93,3 %) nach GridSearch — der Unterschied von einer Blüte ist Rauschen, keine Verschlechterung.

1) Daten und Training — wo man hinschaut

`In [4]` `sns.load_dataset('iris')`	**Zu sehen:** Das Notebook ruft weder `info()` noch `describe()` auf. → **Folgerung:** Iris ist als sauber bekannt (150 × 5, keine fehlenden Werte), es trotzdem zu prüfen ist aber gute Praxis — bei anderen Daten wäre es ein Fehler.

In [6] `sns.pairplot(iris, hue='species')`	**Zu sehen:** Setosa (grün) ist bei petal_length und petal_width völlig getrennt (Blütenblätter kürzer als 2 cm und schmaler als ca. 0,6 cm); versicolor und virginica überlappen leicht (petal_length ca. 4,5–5 cm). → **Folgerung:** Setosa wird fehlerfrei erkannt; die einzig möglichen Verwechslungen sind versicolor ↔ virginica. So sagt die EDA die Struktur der Konfusionsmatrix voraus.
In [7] `sns.kdeplot(setosa[…], …)`	**Zu sehen:** Verteilung sepal_width vs. sepal_length nur für setosa. → **Folgerung:** Illustration ohne Einfluss auf das Modell.
Skalen der Merkmale	**Zu sehen:** Alles in cm, von 0,1 bis 7,9. → **Folgerung:** Ähnliche Bereiche, daher schadet die fehlende Skalierung hier nicht (für SVM grundsätzlich empfohlen: `StandardScaler` in einer Pipeline).
In [9] `train_test_split(X, y, test_size=0.30)`	**Zu sehen:** Kein `random_state` und kein `stratify`; im Test 11 setosa, 17 versicolor, 17 virginica (support in In [16]). → **Folgerung:** 45 Blüten sind ein sehr kleines Testset: 1 Blüte = 2,2 PP Accuracy. Jeder Lauf ergibt einen anderen Split und etwas andere Ergebnisse.
In [19] `GridSearchCV(SVC(), param_grid, refit=True, verbose=2)`	**Zu sehen:** „Fitting 5 folds for each of 16 candidates, totalling 80 fits"; `grid.best_params_` und `grid.best_score_` werden nicht ausgegeben. → **Folgerung:** Wir wissen nicht, welche C- und gamma-Werte gewählt wurden und wie gut die Kreuzvalidierung war — ergänze diese zwei Zeilen. Bei 105 Trainingsbeispielen hat jeder Validierungs-Fold ca. 21 Blüten; die CV-Werte „springen" also in Schritten von ca. 5 PP, und viele Kombinationen liegen gleichauf.



In [6] Pairplot mit `hue='species'` — setosa klar abgesetzt; versicolor und virginica berühren sich bei den Blütenblatt-Maßen.

2) Ergebnisse auf den Testdaten — wie man sie liest

In [15] — Standard-SVC

wahr ↓ / vorh. →	setosa	versic.	virgin.
setosa	**11**	0	0
versicolor	0	**15**	2
virginica	0	0	**17**

In [21] — nach GridSearch

wahr ↓ / vorh. →	setosa	versic.	virgin.
setosa	**11**	0	0
versicolor	0	**14**	3
virginica	0	0	**17**

`In [15] / In [16]` Standard-SVC	**Zu sehen:** Diagonale 11 + 15 + 17 = 43 von 45 (95,6 %); einziger Fehler: 2 versicolor als virginica erkannt. → **So liest du es:** Im Report ist es derselbe Fehler von zwei Seiten: versicolor recall 0,88 (15/17) und virginica precision 0,89 (17/19).
`In [21] / In [22]` nach GridSearch	**Zu sehen:** 3 Fehler (alle versicolor → virginica), 42/45 = 93,3 %. → **Folgerung:** Auf die Frage des Notebooks „Konnten wir uns verbessern?“: nein — es ist aber auch keine echte Verschlechterung. Der Unterschied ist eine Blüte.
Unsicherheit	**Zu sehen:** Bei n = 45 beträgt das 95-%-Intervall der Accuracy etwa ±6–7 PP (berechnet). → **Folgerung:** 95,6 % und 93,3 % sind statistisch nicht unterscheidbar. Das Notebook sagt zu Recht, dass es vor allem eine GridSearch-Übung ist.
Modelle vergleichen `cross_val_score(model, X, y, cv=10)`	**Zu sehen:** Ein einziger Train/Test-Split bei 150 Proben. → **Folgerung:** Bei kleinen Daten Modelle per Kreuzvalidierung vergleichen (Mittelwert ± Standardabweichung über 10 Splits), nicht mit einem einzigen Test.

SVC	GridSearch	Baseline	Unsicherheit
95,6 %	**93,3 %**	**37,8 %**	**±6–7 PP**
43 von 45	42 von 45	berechnet: 17 von 45	berechnet, n = 45

VORSICHT

- Kleines Testset und kein `random_state` → das Ergebnis kann bei jedem Lauf um einige Punkte schwanken.
- GridSearch wählt die Parameter nach der Kreuzvalidierung, nicht nach dem Test — ein schlechteres Ergebnis auf 45 Beispielen heißt nicht, dass die Wahl falsch war.

FAZIT

Ideale, gut getrennte Daten → problemloses Training → sehr gutes Testergebnis (93–96 %). Fehler treten genau dort auf, wo der Pairplot Überlappung gezeigt hat (versicolor/virginica).

B6 K-Means — US-Hochschulen

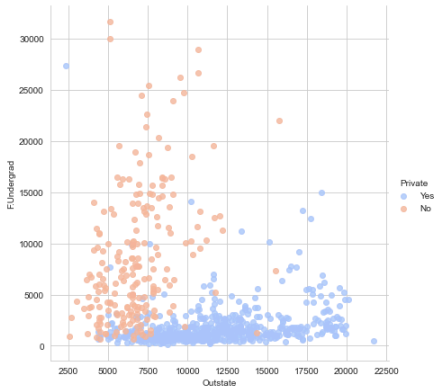
Datei	3-K_Means_Clustering_Projekt-Loesung.ipynb
Aufgabe	ohne Labels 777 Hochschulen in 2 Cluster teilen und prüfen, ob sie der Aufteilung privat/öffentlich entsprechen
Modell	`KMeans(n_clusters=2)`
Typ	unüberwachtes Lernen (Clustering); das Label Private dient nur zur Bewertung
Daten	College_Data: 777 Hochschulen, 17 numerische Merkmale

KURZ GESAGT

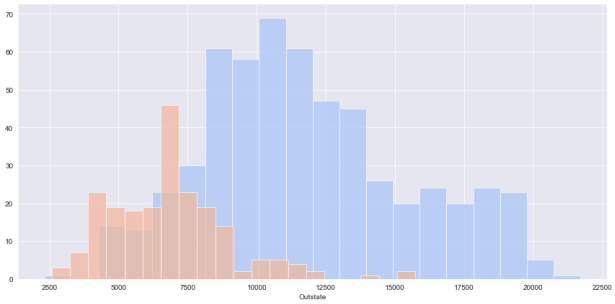
! **Daten**	Vollständig, aber mit falschen Werten (Grad.Rate 118 %, PhD 103 %) und Merkmalsskalen, die sich bis zum Tausendfachen unterscheiden.
! **Training**	K-Means auf 17 unskalierten Merkmalen → die Cluster werden von „Größen“-Merkmalen bestimmt, nicht vom Status privat/öffentlich.
✗ **Ergebnis**	Das Notebook zeigt Accuracy 0,22 — ein Artefakt der Cluster-Nummerierung. Nach korrekter Zuordnung: 0,78 bei Baseline 0,73 → die Cluster entsprechen der Aufteilung privat/öffentlich nur schwach.

1) Daten und Training — wo man hinschaut

`In [26]` `df.info()`	**Zu sehen:** 777 Zeilen (die Einleitung spricht von 770 — kleine Unstimmigkeit), überall non-null; Private als `object` (Yes/No). → **Folgerung:** Keine fehlenden Werte.
`In [27]` `df.describe() – Zeile max`	**Zu sehen:** Grad.Rate max = 118 (eine Abschlussquote > 100 % ist unmöglich) und PhD max = 103 (Anteil Promovierter > 100 %). → **Folgerung:** Zwei Datenfehler. Das Notebook findet den ersten (In [32]: Cazenovia College) und setzt ihn auf 100. Den zweiten übersieht es — es ist die Texas A&M University at Galveston (von mir geprüft). Lektion: min/max JEDER Spalte prüfen, besonders bei Prozentwerten.
`In [27]` `df.describe() – Zeile std`	**Zu sehen:** Apps ≈ 3.870, F.Undergrad ≈ 4.850, Expend ≈ 5.222, Outstate ≈ 4.023, aber S.F.Ratio ≈ 4, Grad.Rate ≈ 17. → **Folgerung:** K-Means rechnet euklidische Abstände, also dominieren Merkmale mit großen Zahlen. Das Notebook skaliert nicht.
`In [29] / In [30]` `lmplot('Outstate', 'F.Undergrad', hue='Private'); hist Outstate`	**Zu sehen:** Klare Trennung: Öffentliche Hochschulen sind groß (F.Undergrad bis 30.000) und für Studierende aus anderen Bundesstaaten günstig (Outstate meist < 10.000); private sind klein und teurer. In [28] (Room.Board vs. Grad.Rate): starke Überlappung. → **Folgerung:** Die EDA zeigt, welche Merkmale die Aufteilung privat/öffentlich tragen: Outstate und F.Undergrad.
`In [33] / In [34]` `df['Grad.Rate'] ['Cazenovia College'] = 100`	**Zu sehen:** Warnung *SettingWithCopyWarning*; In [34] bestätigt die Korrektur (leere Tabelle). → **Folgerung:** In dieser älteren pandas-Version hat es funktioniert. Ab pandas 3.0 (Copy-on-Write) ändert so eine „verkettete" Zuweisung den DataFrame NICHT (von mir geprüft) — verwende `df.loc['Cazenovia College', 'Grad.Rate'] = 100`.
`In [37] / In [38]` `KMeans(n_clusters=2).fit( df.drop('Private', axis=1))`	**Zu sehen:** Keine Skalierung und kein `random_state`. → **Folgerung:** (a) Merkmale mit großen Zahlen dominieren die Cluster; (b) die Nummerierung der Cluster (welcher „0" und welcher „1" ist) ist zufällig und kann sich bei jedem Lauf ändern.



In [29] Outstate vs. F.Undergrad — öffentliche (orange) groß und günstiger, private (blau) klein und teurer.



In [30] Histogramm Outstate nach Private — das beste einzelne Trennmerkmal.

2) Ergebnisse — wie man sie liest

`In [39]` `kmeans.cluster_centers_`	**Zu sehen:** Zwei Zeilen mit je 17 Zahlen (wissenschaftliche Notation). Cluster 1: Apps ≈ 10.363, Accept ≈ 6.551, Enroll ≈ 2.570, F.Undergrad ≈ 13.062; Cluster 0: Apps ≈ 1.813, F.Undergrad ≈ 2.189. Outstate: 10.396 vs. 10.719 — fast identisch! → **So liest du es:** Spalten zwischen den Zeilen vergleichen. K-Means hat die Hochschulen in „groß" und „klein" geteilt, nicht in „teuer privat" und „günstig öffentlich". berechnet: 97 % des Abstands zwischen den Zentren (euklidisch) stammen aus F.Undergrad, Apps, Accept und Expend; Anteil von Outstate ≈ 0 %.

In [43] — wie im Notebook

	Cluster 0	Cluster 1
wahr: öffentlich (0)	**138**	74
wahr: privat (1)	531	**34**

Dieselben Cluster, vertauschte Namen berechnet

	Cluster 1 → „0“	Cluster 0 → „1“
wahr: öffentlich (0)	**74**	138
wahr: privat (1)	34	**531**

`In [43]` `confusion_matrix(df['Cluster'], kmeans.labels_)`	**Zu sehen:** Cluster 0 (669 Hochschulen) = 531 private + 138 öffentliche; Cluster 1 (108 Hochschulen) = 74 öffentliche + 34 private. → **Folgerung:** Cluster 1 sind einfach die großen Hochschulen (überwiegend öffentlich).
`In [44]` `classification_report(…)` `→ accuracy 0.22`	**Zu sehen:** Das Notebook nimmt „Cluster 1 = privat“ an und erhält Accuracy 0,22. → **So liest du es:** K-Means weiß nichts über Labels — die Nummern 0/1 vergibt es zufällig. Bei umgekehrter Zuordnung (Cluster 0 = privat, Cluster 1 = öffentlich): (531 + 74) / 777 = 77,9 % berechnet. Zusatztest: Auf denselben Daten ergab sich bei 20 verschiedenen `random_state` genau diese Matrix 13-mal und die vertauschte 7-mal. Dasselbe Modell kann also „22 %“ oder „78 %“ zeigen.
richtige Bewertung	**Zu sehen:** Beste Zuordnung 77,9 % vs. Baseline „immer privat“ 565/777 = 72,7 % berechnet. → **Folgerung:** Nur +5 PP über der trivialen Antwort. Öffentliche Hochschulen werden zu 35 % erkannt (74 von 212). Adjusted Rand Index (nummerierungsunabhängig: 0 = zufällig, 1 = perfekt) = 0,23 → schwache Übereinstimmung.
`In [44]` „Gar nicht so schlecht…“	**Zu sehen:** Der Kommentar im Notebook lobt das Ergebnis. → **Folgerung:** 22 % ist nicht „gar nicht so schlecht“, sondern vertauschte Labels; nach dem Tausch liegt das Ergebnis kaum über der Baseline. Der Kommentar ist irreführend.

Notebook (In [44])	Nach Zuordnung	Baseline	ARI
22,1 %	**77,9 %**	**72,7 %**	**0,23**
Labels vertauscht	berechnet	„immer privat“	berechnet

ZUSATZTEST (AUSSERHALB DES NOTEBOOKS, DIESELBEN DATEN)

- **Alle 17 Merkmale mit `StandardScaler`:** K-Means teilt die Hochschulen in „elitär/selektiv“ und den Rest (größte Unterschiede der Zentren: Top10perc, Top25perc, PhD, Terminal, Outstate). Die Übereinstimmung mit privat/öffentlich ist noch schlechter: 52 %, ARI ≈ 0.
- **Nur die 2 Merkmale aus der EDA** (Outstate + F.Undergrad, skaliert): 87,4 % Übereinstimmung, ARI 0,52.
- **Lektion:** K-Means findet die dominante Struktur in den Merkmalen, die man ihm gibt — nicht „die, die wir meinen“. Merkmalsauswahl und Skalierung bestimmen das Ergebnis.

VORSICHT

- In echten Anwendungen (ohne Labels) gibt es keine Konfusionsmatrix — das Notebook betont selbst, dass dies nur eine „theoretische“ Kontrolle ist. Die Clusterqualität beurteilt man dann z. B. mit dem *Silhouette-Score*, der Ellbogenmethode (Inertia gegen k) und der Interpretation der Zentren.
- Beim Vergleich von Clustern mit Labels nummerierungsunabhängige Maße (`adjusted_rand_score`) oder die beste Zuordnung verwenden.

FAZIT

Die Daten brauchten Bereinigung (2 unmögliche Werte) sowie Skalierung und Merkmalsauswahl. Die Cluster spiegeln die Größe der Hochschulen wider, nicht ihren Status. 0,22 ist ein Nummerierungsartefakt; tatsächlich sind es 0,78 bei einer Baseline von 0,73 — schwach.

B7 NLP — Yelp-Bewertungen

Datei	3-Nlp_Projekt-Loesung.ipynb
Aufgabe	anhand des Bewertungstexts 1 Stern vs. 5 Sterne erkennen
Modelle	(1) `CountVectorizer` + `MultinomialNB`; (2) Pipeline: `CountVectorizer` + `TfidfTransformer` + `MultinomialNB`
Typ	Textklassifikation (binär), unausgewogene Klassen
Daten	10.000 Yelp-Bewertungen; für die Aufgabe 4.086 (749 × 1★, 3.337 × 5★)

KURZ GESAGT

! **Daten**	Vollständig (keine fehlenden Werte), aber unausgewogen: nur 18 % der Bewertungen haben 1★.
! **Training**	Modell 1 korrekt (mit kleinem Leck: Vokabular auf allen Daten gebaut). Modell 2 (TF-IDF) degeneriert.
! **Testergebnis**	Modell 1: 92,6 % bei Baseline 81,4 %, übersieht aber 30 % der 1★-Bewertungen. Modell 2: immer „5★" = genau die Baseline — nutzlos, und der Report im Notebook verdeckt das.

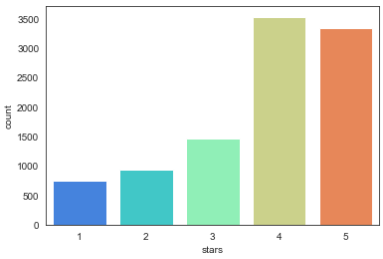
1) Daten und Training — wo man hinschaut

`In [4]` `yelp.info()`	**Zu sehen:** 10.000 Zeilen, überall non-null; text als `object`. → **Folgerung:** Keine fehlenden Werte.
`In [5]` `yelp.describe()`	**Zu sehen:** Stars im Mittel 3,78; cool/useful/funny — max 77/76/57 bei einem Median von 0–1. → **Folgerung:** Die Bewertungen sind ins Positive verschoben; die Votes stark schief (im Modell ohnehin ungenutzt).
`In [8] / In [9]` `FacetGrid(col='stars') / boxplot text length`	**Zu sehen:** Ähnliche Histogrammformen, fast gleiche Mediane der Textlänge für 1–5★, viele Ausreißer. → **Folgerung:** Die Textlänge unterscheidet die Bewertungen kaum — kein gutes Merkmal. (Unterschiedliche Balkenhöhen im FacetGrid sind einfach unterschiedliche Klassengrößen.)
`In [10]` `sns.countplot(x='stars')`	**Zu sehen:** 4★ und 5★ dominieren (ca. 3.500 und 3.300), 1★ ist am seltensten (ca. 750). → **Folgerung:** In der Aufgabe 1★ vs. 5★ sind die Klassen unausgewogen: 749 vs. 3.337, also 18,3 % 1★ (berechnet). Baseline „immer 5★" ≈ 81 %.
`In [11] → In [13]` `groupby('stars').mean() → corr() → heatmap`	**Zu sehen:** Korrelationen um −0,94 und 0,89. → **Folgerung:** Achtung: berechnet auf 5 Zeilen (Mittelwerte für 1…5 Sterne), nicht auf 10.000 Bewertungen. Bei n = 5 können so hohe Korrelationen zufällig entstehen; es ist eine Korrelation von Gruppenmitteln, kein Zusammenhang zwischen einzelnen Bewertungen (ökologischer Fehlschluss). Aus dieser Heatmap keine Schlüsse über das Modell ziehen.
`In [14]` `yelp[(yelp.stars==1) | (yelp.stars==5)]`	**Zu sehen:** Nur 1★ und 5★ bleiben übrig. → **Folgerung:** 4.086 Bewertungen zum Klassifizieren.
`In [17] → In [19]` `X = cv.fit_transform(X), dann train_test_split`	**Zu sehen:** Das Vokabular (Zusatztest: 19.183 Wörter) wurde auch aus den Testbewertungen gebaut. → **Folgerung:** Kleines Datenleck. Korrekt: erst splitten, dann `fit` nur auf den Trainingsdaten (so arbeitet die Pipeline im zweiten Teil). Zusatztest: In der korrekten Reihenfolge ist das Ergebnis fast identisch ([[155, 73], [15, 983]], 92,8 %) — hier hat das Leck nichts verändert, die Gewohnheit ist trotzdem wichtig.

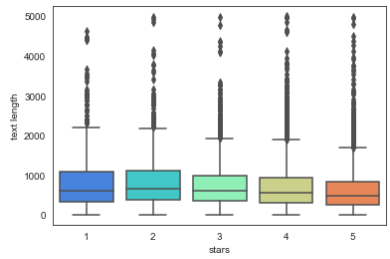
In [19]

```
train_test_split(X, y,
test_size=0.3,
random_state=101)
```

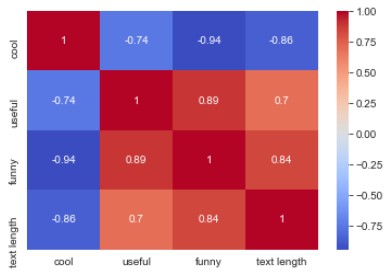
Zu sehen: Test = 1.226 Bewertungen (228 × 1★, 998 × 5★ — support in In [24]). → **Folgerung:** Anteile wie im Gesamtdatensatz; reproduzierbar.



In [10] Anzahl der Bewertungen pro Sternzahl — 1★ ist die kleinste Gruppe.



In [9] Textlänge pro Sternzahl — Mediane fast gleich.



In [13] Korrelations-Heatmap — nur aus 5 Mittelwerten berechnet.

2) Ergebnisse auf den Testdaten — wie man sie liest

In [24] — Modell 1: Count + NB

	vorh. 1★	vorh. 5★
wahr: 1★	**159**	69
wahr: 5★	22	**976**

In [31] — Modell 2: TF-IDF + NB

	vorh. 1★	vorh. 5★
wahr: 1★	**0**	228
wahr: 5★	**0**	**998**

Spalte nur mit Nullen: Das Modell wählt nie 1★.

In [24]

```
Modell 1 –
Konfusionsmatrix
```

Zu sehen: Diagonale 159 + 976 = 1.135 Treffer von 1.226 = 92,6 %. 22 5★-Bewertungen als 1★ erkannt; 69 1★-Bewertungen als 5★. → **So liest du es:** Zeilen = Wahrheit, Spalten = Vorhersage, Reihenfolge der Klassen 1★, 5★.

In [24]

```
Modell 1 –
classification_report
```

Zu sehen: 1★: precision 0,88, recall 0,70; 5★: precision 0,93, recall 0,98; accuracy 0,93; macro avg F1 0,87, weighted 0,92. → **So liest du es:** Wenn das Modell „1★" sagt, stimmt es in 88 % (159 von 181). Aber von 228 echten negativen Bewertungen findet es 159 — jede dritte wird übersehen. Geht es darum, unzufriedene Kunden zu finden, ist genau der Recall von 1★ entscheidend. Der Unterschied macro (0,87) vs. weighted (0,92) verrät die schwächere kleine Klasse.

Baseline

Zu sehen: 998 / 1.226 = 81,4 % berechnet. → **Folgerung:** Modell 1 ist 11 PP besser — es hat wirklich gelernt. Balanced accuracy 0,84 berechnet.

Modell 1	Baseline	Recall 1★	Modell 2
92,6 %	**81,4 %**	**0,70**	**81,4 %**
Accuracy	berechnet	69 übersehen	= Baseline, Recall 1★ = 0

In [31]

```
Modell 2 – Matrix [[0,
228], [0, 998]]
```

Zu sehen: Die erste Spalte enthält nur Nullen. → **So liest du es:** Das Modell sagt NIE 1★ voraus. Ein konstanter Klassifikator: accuracy = 998 / 1.226 = 81,4 % = genau die Baseline; balanced accuracy 0,50 berechnet = Rateniveau.

In [31]

```
classification_report(…,
labels=np.unique(predicti
ons))
```

Zu sehen: Nur die Zeile der Klasse 5 (precision 0,81, recall 1,00, f1 0,90) und eine seltsame Zeile „micro avg". → **Folgerung:** Der Report verdeckt das Problem! `labels=np.unique(predictions)` beschränkt den Report auf die vorhergesagten Klassen — und vorhergesagt wurde nur 5★. Der vollständige Report (ohne `labels`) zeigte für 1★ precision 0 und recall 0 (mit *UndefinedMetricWarning*). Regel: zuerst die Konfusionsmatrix.

WARUM HAT TF-IDF DAS MODELL „KAPUTT“ GEMACHT? (ZUSATZTEST, DIESELBEN DATEN)

Das Notebook stellt nur fest, dass TF-IDF das Ergebnis verschlechtert hat. Die Ursache liegt in der Kombination von TF-IDF mit `MultinomialNB` und der Glättung `alpha=1`:

- TF-IDF-Gewichte sind kleine Brüche: Pro Bewertung summieren sie sich auf ca. 7 statt auf ca. 113 Wortzählungen.
- Die gesamte TF-IDF-„Masse“ aller 1★-Bewertungen im Training beträgt ca. 4.100, die Laplace-Glättung addiert aber 1 zu jedem der 16.233 Wörter des Vokabulars — sie ist also etwa 4-mal größer als die Daten dieser Klasse.
- Die Wortverteilung der Klasse 1★ wird fast zu einer Gleichverteilung „verwischt“. Dann sprechen sogar neutrale Wörter für 5★ — so war es bei 99,8 % der Testbewertungen.
- Beleg: `fit_prior=False` (Klassenanteile ausgeschaltet) hilft nicht — 2 von 228 1★ erkannt; weniger Glättung hilft: `alpha=0.01` → 121 von 228. Am besten funktionierte TF-IDF + `LogisticRegression(class_weight='balanced')`: 93,4 % Accuracy, Recall 1★ = 0,88 (201 von 228), balanced accuracy 0,91.

VORSICHT

- Bei unausgewogenen Klassen die Accuracy immer mit der Baseline (hier 81 %) vergleichen und auf den Recall der kleinen Klasse sowie macro avg schauen.
- `classification_report` nie mit `labels` auf die vorhergesagten Klassen beschränken.
- Textverarbeitung (CountVectorizer, TF-IDF) nur auf den Trainingsdaten anpassen — die Pipeline erledigt das automatisch.

FAZIT

Gute, aber unausgewogene Daten. Modell 1 (Count + NB) funktioniert wirklich: 93 % bei 81 % Baseline, übersieht aber jede dritte negative Bewertung. Modell 2 (TF-IDF + NB mit `alpha=1`) ist ein konstantes Modell — seine 81 % sind nur die Mehrheitsklasse.

B8 Keras — LendingClub (neuronales Netz)

Datei	04-Keras-Projekt-Uebung-Loesung.ipynb
Aufgabe	vorhersagen, ob ein Kreditnehmer den Kredit zurückzahlt (`loan_repaid`: 1 = Fully Paid, 0 = Charged Off)
Modell	Keras `Sequential`: Dense 78 → 39 → 19 → 1, Dropout 0,2, Sigmoid, `binary_crossentropy`, Adam, 25 Epochen
Typ	binäre Klassifikation, unausgewogene Klassen (80/20)
Daten	396.030 Kredite, 27 Spalten; Modell auf einer 10-%-Stichprobe trainiert (39.522 Zeilen)

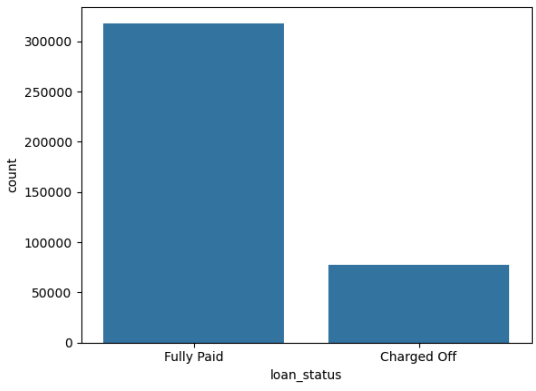
Verweise: Die Datei enthält Ausgaben aus zwei Läufen, daher wiederholen sich manche Nummern In [n] — orientiere dich am Codeausschnitt und an der Abschnittsüberschrift (Abschnitt 1–3).

KURZ GESAGT

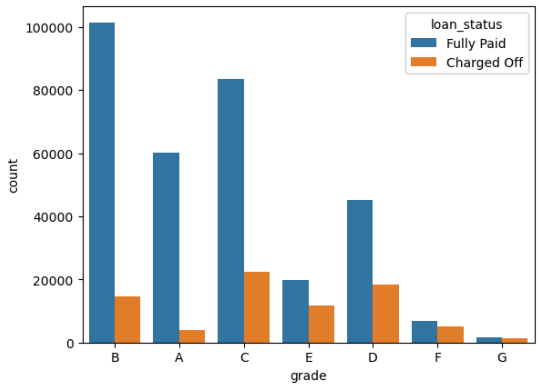
! **Daten**	Die Rohdaten brauchten viel Arbeit: fehlende Werte (bis 9,5 %), 15 Textspalten, doppelte Merkmale und ein „leckendes“. Das Notebook bereitet das vorbildlich auf. Klassen 80/20.
✓ **Training**	Kurve loss/val_loss: schnelle Konvergenz; ab ca. Epoche 5 ist val_loss flach, später steigt er leicht → milde Überanpassung.
! **Testergebnis**	Accuracy 88,1 % bei Baseline 80,1 % (+8 PP), aber der Recall für „Charged Off“ beträgt nur 0,46 → das Modell erkennt weniger als die Hälfte der faulen Kredite.

1a) Daten — Abschnitt 1: Explorative Datenanalyse

`In [8]` `df.info()`	**Zu sehen:** 396.030 Zeilen, 27 Spalten; *Non-Null Count* < 396.030 bei: emp_title, emp_length, title, revol_util, mort_acc, pub_rec_bankruptcies; 15 `object`-Spalten. → **Folgerung:** Fehlende Werte und viel Text — die Daten müssen vorbereitet werden.
`In [9] / In [17]` `countplot(x='loan_status'); groupby('loan_status')['loan_amnt'].describe()`	**Zu sehen:** Fully Paid 318.357 vs. Charged Off 77.673 (80,4 % / 19,6 % berechnet). → **Folgerung:** Unausgewogene Klassen; Baseline „zahlt immer zurück" ≈ 80 %. Mit dieser Zahl muss die Accuracy verglichen werden.
`In [16] / In [17]` `boxplot(x='loan_status', y='loan_amnt')`	**Zu sehen:** Median 14.000 (Charged Off) vs. 12.000 (Fully Paid); die Boxen überlappen stark. → **Folgerung:** Der Kreditbetrag ist ein schwacher Prädiktor.
`In [15]` `scatterplot(x='installment', y='loan_amnt')`	**Zu sehen:** Fast linearer Zusammenhang (die Rate folgt aus Betrag, Zins und Laufzeit). → **Folgerung:** Doppelte Information — das Notebook fragt direkt danach. Für ein Netz kein Fehler, aber das Merkmal bringt nichts Neues.
`In [20] und folgende` `countplot(x='grade', hue='loan_status'); sub_grade; F und G`	**Zu sehen:** Der Anteil Charged Off steigt systematisch von A nach G; bei F und G ist es fast 50/50. → **Folgerung:** Grade/sub_grade ist ein starker Prädiktor (praktisch die Risikoeinstufung von LendingClub).
`In [27]` `df.corr()['loan_repaid'].sort_values().drop('loan_repaid').plot(kind='bar')`	**Zu sehen:** Am stärksten int_rate ≈ −0,25; alle anderen |r| < 0,1. → **Folgerung:** Einzelne numerische Merkmale sind schwach — man braucht die Kombination vieler Merkmale (das rechtfertigt ein neuronales Netz).
`In [11] / In [12]` `df.corr() → ValueError: could not convert string to float: ' 36 months'`	**Zu sehen:** Fehler statt Heatmap. → **Folgerung:** Stammt aus dem neueren Lauf, nicht aus den Daten: Seit pandas 2.0 überspringt `corr()` keine Textspalten mehr. Lösung: `df.corr(numeric_only=True)`.



In [9] loan_status — ca. 80 % Fully Paid: Baseline ≈ 80 %.



In [20] grade nach loan_status — der Anteil Charged Off steigt von A bis G.

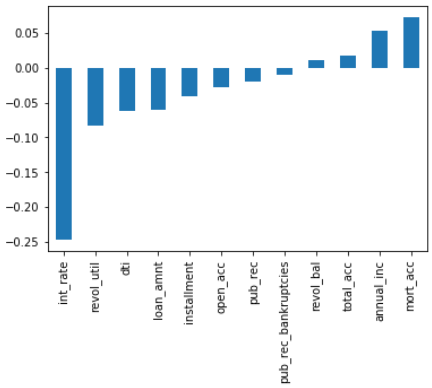
1b) Daten — Abschnitt 2: Daten-Preprocessing

`In [30] / In [31]` `df.isnull().sum() / in %`	**Zu sehen:** Mort_acc 9,5 %, emp_title 5,8 %, emp_length 4,6 %, title 0,4 %, pub_rec_bankruptcies 0,14 %, revol_util 0,07 %. → **Folgerung:** Entscheidungen im Notebook: emp_title entfernt (In [33]: 173.105 eindeutige Werte — nicht kodierbar); emp_length entfernt (siehe unten); title entfernt (In [48]/[49]: beschreibt dasselbe wie purpose); mort_acc mit dem Mittelwert je total_acc-Gruppe aufgefüllt (In [53]: höchste Korrelation 0,38) — sinnvoll; revol_util und pub_rec_bankruptcies: `dropna()` (< 0,5 % der Zeilen).

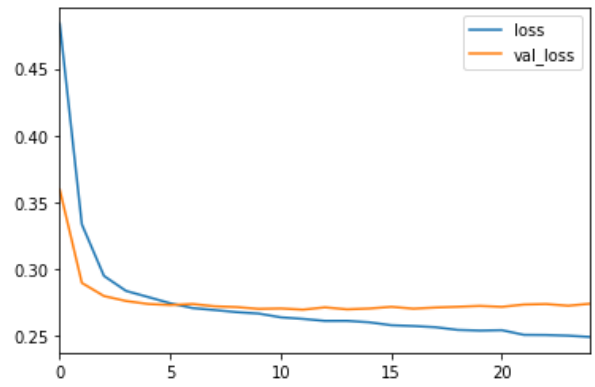
In [44] / In [45] `emp_len = emp_co / emp_fp`	**Zu sehen:** Werte 0,23–0,26 in jeder Kategorie der Beschäftigungsdauer. → **So liest du es:** Das ist das Verhältnis Charged Off zu Fully Paid (wie viele Ausfälle pro zurückgezahltem Kredit), kein Prozentsatz — obwohl die Aufgabe von „Prozentanteil" spricht. Anteil der Ausfälle = r/(1+r) ≈ 18–21 % berechnet in allen Gruppen; flaches Profil = Merkmal ohne Information, zu Recht entfernt.
In [64] → In [77] `Kodierung der Kategorien`	**Zu sehen:** Term → 36/60; grade entfernt (in sub_grade enthalten); Dummies: sub_grade, verification_status, application_type, initial_list_status, purpose, home_ownership (NONE/ANY → OTHER); zip_code aus address → Dummies; earliest_cr_line → Jahr. → **Folgerung:** Am Ende ist nur noch loan_status Text (In [77]), der danach als Duplikat von loan_repaid entfernt wird. Das Modell erhält 78 Merkmale — passt zu `Dense(78)`.
In [75] `df.drop('issue_d', axis=1)`	**Zu sehen:** Begründung: „Datenüberlauf" (Datenleck). → **Folgerung:** Vorbildliches Denken: Bei der Kreditentscheidung kennt man das Auszahlungsdatum noch nicht — das Merkmal wäre erst „hinterher" verfügbar.
In [28] `df.head() – Spalte address`	**Zu sehen:** „Mendozaberg, OK 22690", „Loganmouth, SD 05113" und „New Sabrina, WV 05113" — dieselbe Postleitzahl in zwei Bundesstaaten. → **Folgerung:** Die Adressen sind synthetisch. Eine etwaige Vorhersagekraft von zip_code ist ein Artefakt dieses Kursdatensatzes, keine reale Gesetzmäßigkeit. Prüfen: `df.groupby('zip_code')['loan_repaid'].mean()`.
In [82] `print(len(df)) → 39522`	**Zu sehen:** Die Zeile `df.sample(frac=0.1, …)` ist jetzt auskommentiert, aber 39.522 zeigt, dass im Lauf, aus dem die Ergebnisse stammen, eine 10-%-Stichprobe verwendet wurde. → **Folgerung:** 395.219 × 0,1 ≈ 39.522; Test 20 % = 7.905 — passt zum support im Report berechnet.
In [88] / In [89] `scaler.fit_transform(X_train); scaler.transform(X_test)`	**Zu sehen:** MinMaxScaler nur auf den Trainingsdaten angepasst. → **Folgerung:** Korrekt, ohne Leck (das Notebook begründet es ausdrücklich).

1c) Training — wie es lief

In [24] `Dense(78) … Dense(1, activation='sigmoid'); compile(loss='binary_crossentropy')`	**Zu sehen:** Trichter-Architektur mit Dropout 0,2 nach jeder versteckten Schicht. → **Folgerung:** Standard für binäre Klassifikation; Dropout begrenzt Overfitting.
In [25] `model.fit(…, epochs=25, batch_size=256, validation_data=(X_test, y_test))`	**Zu sehen:** Das Testset dient zugleich als Validierungsset. → **Folgerung:** Wer die Kurven beobachtet (und erst recht mit Early Stopping), macht den Test abhängig. Korrekt: eigenes Validierungsset (z. B. `validation_split=0.1`), Test nur einmal ganz am Ende.
In [97] `losses[['loss','val_loss']].plot()`	**Zu sehen:** Loss fällt stetig von ca. 0,49 auf ca. 0,25; val_loss fällt schnell auf ca. 0,27 (ca. 5 Epochen), bleibt dann flach und steigt ab ca. Epoche 15 leicht (bis ca. 0,274), während loss weiter fällt. → **So liest du es:** Eine wachsende Lücke zwischen den Kurven = beginnende Überanpassung (das Netz lernt Details der Trainingsdaten, die nicht verallgemeinern). Anfangs ist val_loss < loss — das ist normal: Dropout wirkt nur im Training, und loss ist ein Mittel über die ganze Epoche. Etwa 5–10 Epochen hätten gereicht; `EarlyStopping(monitor='val_loss', patience=3)` hätte automatisch gestoppt. Die Überanpassung ist mild (Lücke ca. 0,025), die Ergebnisse bleiben verlässlich.



In [27] Korrelation der Merkmale mit loan_repaid — nur int_rate deutlich (≈ −0,25).



In [97] loss (Training) vs. val_loss — val_loss ab ca. Epoche 5 flach, die Lücke wächst.

2) Ergebnisse auf den Testdaten — wie man sie liest

In [101] — confusion_matrix(y_test, predictions)

	vorh. 0	vorh. 1
wahr: 0 Charged Off	**723**	847
wahr: 1 Fully Paid	95	**6240**

847 = faule Kredite als gut eingestuft (der teuerste Fehler für den Kreditgeber).

Accuracy **88,1 %** 6.963 von 7.905	Baseline **80,1 %** berechnet
Recall Charged Off **0,46** 847 übersehen	Balanced acc. **0,72** berechnet

`In [101]` `Klassenkodierung`	**Zu sehen:** Klasse 0 = Charged Off (nicht zurückgezahlt), 1 = Fully Paid; Zeilen = Wahrheit, Spalten = Vorhersage. → **Folgerung:** Achtung: Hier ist „Klasse 0" die wichtigere (Risiko).
`In [100]` `classification_report(y_test, predictions)`	**Zu sehen:** Klasse 0: precision 0,88, recall 0,46; Klasse 1: precision 0,88, recall 0,99; accuracy 0,88; macro avg recall 0,72 / F1 0,77; weighted 0,88 / 0,87. → **So liest du es:** Das Modell erkennt 46 % der faulen Kredite (723 von 1.570); wenn es „zahlt nicht" warnt, stimmt es in 88 %. Gute Kredite erkennt es fast alle (99 %). Unterschied macro vs. weighted = Schwäche bei der kleineren Klasse.
Baseline	**Zu sehen:** 6.335 / 7.905 = 80,1 % (berechnet). → **Folgerung:** Modell 88,1 % = +8 PP. Eine echte, aber moderate Verbesserung.
Geschäftliche Interpretation	**Zu sehen:** (berechnet): Würde die Bank nur Kunden mit der Vorhersage „1" einen Kredit geben, fielen von 7.087 bewilligten Krediten 847 aus — 12,0 % statt 19,9 % ohne Modell. Abgelehnt würden 818 Anträge, davon 95 (11,6 %) gute Kunden. → **Folgerung:** In dieser Sprache sollte man das Ergebnis beschreiben: Um wie viel sinkt die Ausfallquote, und wie viele gute Kunden verliert man?
`In [99]` `model.predict_classes(X_test)`	**Zu sehen:** Standardschwelle 0,5. → **Folgerung:** Bei unausgewogenen Klassen kann man sie verschieben (z. B. „zahlt zurück" erst ab p > 0,6–0,7): mehr erkannte faule Kredite, dafür mehr abgelehnte gute. Die Wahl hängt von den Kosten beider Fehler ab — hilfreich ist die Precision-Recall-Kurve.
`In [102] → In [104]` `new_customer … predict … loan_repaid`	**Zu sehen:** Das Modell sagt 1 vorher, die Wahrheit ist 1. → **Folgerung:** Ein einzelnes Beispiel beweist nichts — ein Modell „immer 1" läge hier auch richtig (und das in 80 % der Fälle). Außerdem stammt dieser Kunde aus df, war also mit ca. 80 % Wahrscheinlichkeit im Trainingsset — kein „neuer" Kunde.

TECHNISCHER ZUSTAND DER DATEI (WICHTIG BEIM ERNEUTEN AUSFÜHREN)

- Der neuere Lauf (14.10.2026, Python 3.13.9) umfasst In [1]–[21] (EDA) und In [22]–[29] (Modellzellen am Ende); die übrigen Ausgaben — u. a. Loss-Plot, Report und Matrix — stammen aus einem älteren Lauf.
- In [11]/[12] `df.corr()` → ValueError: ab pandas 2.0 `df.corr(numeric_only=True)` verwenden.
- In [25] `fit` → NameError (X_train existiert nicht); In [26] → NameError (train_test_split erst in In [27] importiert); In [29] → KeyError 'loan_repaid' (die gesamte Datenvorbereitung wurde übersprungen). Lösung: *Kernel → Restart & Run All*, also alles der Reihe nach.
- `model.predict_classes` gibt es im neuen TensorFlow nicht mehr (seit 2.6 entfernt): `(model.predict(X_test) > 0.5).astype(int)`.
- `df.groupby('total_acc').mean()['mort_acc']` → ab pandas 2.0: `df.groupby('total_acc')['mort_acc'].mean()`.

FAZIT

Die Daten brauchten viel Arbeit (fehlende Werte, Kategorien, Leck) — das Notebook erledigt das vorbildlich. Das Training lief korrekt, mit milder Überanpassung ab ca. Epoche 10–15. Ergebnis: +8 PP über der Baseline, sehr gute Erkennung guter Kredite, aber nur 46 % der faulen erkannt. Vor einem praktischen Einsatz müsste man die Schwelle oder die Klassengewichte anpassen.

B9 Alle Modelle im Vergleich + wichtigste Lektionen

Alle Ergebnisse auf den Testdaten in einer Übersicht. Entscheidend ist der Vergleich mit der **Baseline** — dem Ergebnis eines „Modells“, das immer die häufigste Klasse wählt.

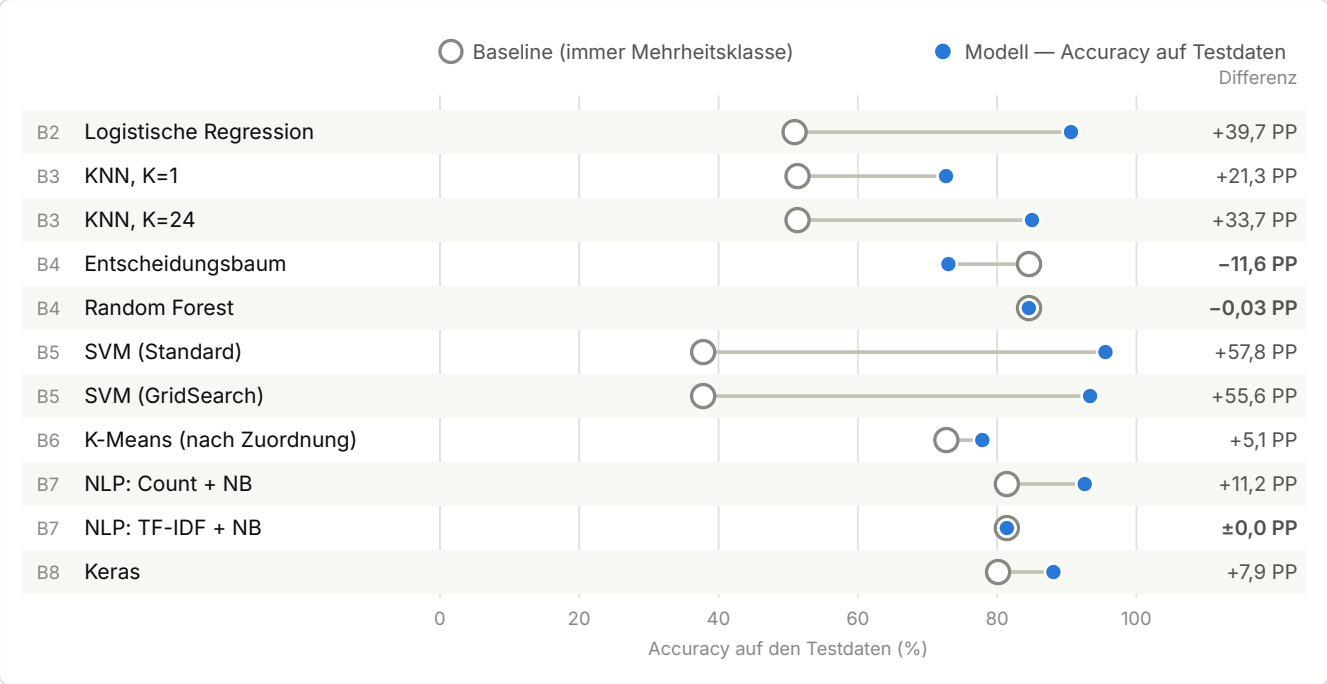


Diagramm: Accuracy auf den Testdaten (gefüllter Punkt) vs. Baseline (leerer Kreis); rechts die Differenz in Prozentpunkten. K-Means — nach Zuordnung der Clusternamen (berechnet). Die lineare Regression hat keine Accuracy und steht daher nur in der Tabelle.

	NOTEBOOK	DATEN	ERGEBNIS AUF TESTDATEN	BASELINE	BALANCED ACC.	URTEIL
B1	**Lineare Regression** Regression / LinearRegression	✓ sauber, synthetisch	RMSE 8,93 $ · R^2 ≈ 0,99	RMSE ≈ 79 $	–	✓ sehr gut
B2	**Logistische Regression** Klassifikation / LogisticRegression	✓ ausgewogen, trennbar	90,6 %	50,9 %	0,91	✓ sehr gut
B3	**KNN** Klassifikation / KNN, K=24	! Klassen überlappen	85,0 % (K=1: 72,7 %)	51,3 %	0,85	✓ gut, leicht optimistisch
B4	**Entscheidungsbaum** Klassifikation	! 16 % Klasse 1, schwaches Signal	73,0 % · recall(1) 0,24	84,6 %	0,53	✗ unter der Baseline
B4	**Random Forest** Klassifikation	! wie oben	84,55 % · recall(1) 0,02	84,59 %	0,51	✗ = Baseline
B5	**SVM** Klassifikation, 3 Klassen / SVC	✓ vorbildlich	95,6 % / 93,3 %	37,8 %	0,96 / 0,94	✓ sehr gut
B6	**K-Means** Clustering	! Fehler, Skalen	77,9 % nach Zuordnung (Notebook: 22,1 %)	72,7 %	0,64	✗ schwach
B7	**NLP Modell 1** Text / Count + NB	! unausgewogen	92,6 % · recall(1★) 0,70	81,4 %	0,84	✓ gut
B7	**NLP Modell 2** Text / TF-IDF + NB	! wie oben	81,4 % · recall(1★) 0	81,4 %	0,50	✗ konstantes Modell
B8	**Keras** neuronales Netz	! Lücken, 80/20	88,1 % · recall(Charged Off) 0,46	80,1 %	0,72	! mäßig

Die wichtigsten Lektionen aus 8 Notebooks

1. **Bevor du dich über die Accuracy freust, berechne die Baseline.** Random Forest (84,55 %) und NLP mit TF-IDF (81,4 %) erreichen genau so viel wie ein „Modell“, das immer die Mehrheitsklasse wählt (B4, B7).
2. **Erst die Konfusionsmatrix, dann der Report.** Eine Spalte nur mit Nullen = eine Klasse, die das Modell nie vorhersagt (B7). Den Report kann man mit `labels` „schönen“ — die Matrix nicht.
3. **Bei unausgewogenen Klassen zählt die kleine Klasse:** ihr Recall und ihre Precision sowie macro avg / balanced accuracy (B4, B7, B8).
4. `describe()` **→ min und max jeder Spalte.** Unmögliche Werte (Grad.Rate 118, PhD 103) fallen sofort auf (B6).
5. **Abstandsbasierte Verfahren (KNN, K-Means, SVM, Netze) brauchen Skalierung** — und Scaler, Vectorizer und Imputation passt man nur auf den Trainingsdaten an (B3, B7 vs. B8).
6. **Das Testset ist eine einmalige Prüfung:** Wähle darauf weder K noch Hyperparameter noch die Epochenzahl (B3, B8).
7. **Kleines Testset = große Unsicherheit.** Ein Unterschied von einem einzigen Beispiel bei 45 ist Rauschen (B5).
8. **Beim Clustering bedeuten Cluster-Nummern nichts** — mit Labels über die beste Zuordnung oder den ARI vergleichen (B6).
9. **Die EDA kündigt das Ergebnis an:** getrennte Wolken im Pairplot → hohe Trefferquote (B2, B5); überlappende Verteilungen → schwaches Modell (B4).
10. **Eine einzelne richtige Vorhersage beweist nichts** („neuer Kunde“ in B8).

B10 Technische Hinweise für neue Bibliotheksversionen

Ein Teil der Ausgaben in den Notebooks entstand mit älteren Bibliotheksversionen, in den Metadaten mehrerer Dateien steht aber bereits eine Python-3.13-Umgebung. In neuen Versionen liefern folgende Stellen einen Fehler oder ein anderes Ergebnis.

PROBLEM	WO	LÖSUNG
`df.corr()` oder `groupby(…).mean()` auf Daten mit Textspalten → ValueError / TypeError (pandas ≥ 2.0)	Keras In [11], [12], [27], [53], [54]; NLP In [11]	`df.corr(numeric_only=True)`, `groupby(…).mean(numeric_only=True)` oder Spalte wählen: `groupby('total_acc')['mort_acc'].mean()`
Verkettete Zuweisung `df['Spalte']['Zeile'] = …` ändert nichts (pandas ≥ 3.0, Copy-on-Write)	K-Means In [33]	`df.loc['Cazenovia College', 'Grad.Rate'] = 100`
Positionsargumente in seaborn (`lmplot('a', 'b', …)`, `kdeplot(x, y)`, `shade=`) → Fehler ab seaborn 0.12	K-Means In [28], [29]; SVM In [7]	`sns.lmplot(x='Room.Board', y='Grad.Rate', data=df, …)`; `sns.kdeplot(x=…, y=…, fill=True, thresh=0.05)`
`sns.distplot` veraltet	Keras In [10]	`sns.histplot(df['loan_amnt'], bins=40)`
`model.predict_classes` entfernt (TensorFlow ≥ 2.6)	Keras In [99], [103]	`(model.predict(X) > 0.5).astype(int)`
Anderes Layout von `classification_report` („avg / total“)	KNN In [30], [33]	nur das Format einer älteren Version; heute: accuracy / macro avg / weighted avg
Kein `random_state` → andere Zahlen bei jedem Lauf	KNN In [23]; SVM In [9]; K-Means In [37]; Bäume In [18], [25]	`random_state=…` im Split und im Modell setzen, wenn du Ergebnisse vergleichen willst
Zellen nicht der Reihe nach ausgeführt → NameError / KeyError	Keras In [25], [26], [29]	*Kernel → Restart & Run All*